



UNIVERSITÀ DEGLI STUDI ROMA TRE

Dipartimento di Ingegneria Civile, Informatica e delle
Tecnologie Aeronautiche

Corso di Laurea Magistrale in Ingegneria Informatica

Tesi Di Laurea

Automated Verification of MANRS Compliance
through Network Emulation for Multi-Router
Autonomous Systems

Candidato

Lorenzo Sacco

Matricola 580515

Relatore

Dr. Tommaso Caiazzì

Correlatore

Dr. Mariano Scazzariello

Dr. Antonio Prado

Anno Accademico 2025/2026

Abstract

The Border Gateway Protocol (BGP) forms the foundation of inter-domain routing and enables communication between Autonomous Systems (ASes) across the Internet. Despite its widespread adoption, BGP was originally designed with limited security mechanisms, making the global routing infrastructure vulnerable to threats such as route leaks and IP address spoofing. To improve routing security, the Mutually Agreed Norms for Routing Security (MANRS) initiative defines sets of operational best practices, organized into distinct action programs for different classes of participants, including network operators, Internet Exchange Points (IXPs), CDN, cloud providers and equipment vendors, who are encouraged to implement and validate them. This thesis focuses specifically on the network operators program, as it is the most directly relevant to the routing security mechanisms discussed in the following chapters.

This thesis focuses on the automation of MANRS compliance verification through network emulation. The work builds upon ROSE-T, an open-source framework capable of reconstructing an emulated network environment from router configurations and evaluating the behaviour of an Autonomous System against selected MANRS actions. While the original framework demonstrated the feasibility of this approach, it was limited to scenarios in which the candidate Autonomous System consisted of a single router.

The main contribution of this thesis is the extension of ROSE-T to support realistic multi-router Autonomous Systems. The proposed solution introduces a topology reconstruction mechanism capable of identifying and modelling internal

BGP sessions, extends the emulation pipeline to instantiate multiple candidate routers while preserving their original configurations and enhances the adaptation process to support modern FRRouting-based laboratories. Additional improvements include automated generation of MANRS-oriented test scenarios and the integration of local routing information to reproduce realistic Internet environments.

As routing misconfigurations can have significant operational and security consequences, the ability to validate routing policies before their deployment is of paramount importance. This work enables network operators to assess the compliance of their routing configurations within realistic emulated environments. The experimental results demonstrate that the proposed approach significantly broadens the applicability of ROSE-T to realistic network deployments while maintaining reliable verification capabilities and providing network operators with a practical environment for safely evaluating routing security before applying configuration changes to production infrastructures.

Contents

Abstract	ii
Contents	iv
List of Figures	vii
1 Introduction	1
2 Preliminaries	4
2.1 Border Gateway Protocol	4
2.1.1 Inter-domain Relationships	7
2.1.2 Internet Routing Registries	9
2.2 Routing Security Threats	11
2.2.1 Route leaks	12
2.2.2 Spoofed Packets	15
2.3 MANRS	19
2.3.1 Action 1 – Filtering	22
2.3.2 Action 2 – Anti-Spoofing	23
2.3.3 Action 3 – Coordination	24
2.3.4 Action 4 – Global Validation	24
2.4 Kathará	25
2.4.1 Kathará Python API	29

3	State of the Art	31
3.1	Configuration Verifiers	32
3.2	Real-time Monitoring	34
3.3	Emulation and Testing	35
4	ROSE-T	37
4.1	Original architecture	39
4.1.1	VRNetLab	43
4.2	Verification pipeline	44
4.3	Verification Actions	47
4.3.1	Route Leak	48
4.3.2	Anti-Spoofing	50
4.3.3	Global Information	53
4.4	Limitations	55
4.4.1	Dependence on External Routing Information	56
4.4.2	Limited Vendor Support	57
4.4.3	Single Router Assumption	57
5	Extending ROSE-T	59
5.1	FRR support	60
5.1.1	Motivation	60
5.1.2	Configuration Parsing	61
5.1.3	Startup File Specification	62
5.2	Global Information File	63
5.2.1	Motivation	63
5.2.2	Implementation Details	66
5.3	Multi-router AS support	68
5.3.1	Topology Builder	69
5.3.2	Actions Redesign	73
5.4	ROSET-Adapter	78

Contents	vi
5.5 User interaction	80
5.6 Conclusions and comments	83
6 Tests and Results	86
6.1 Evaluation Methodology	86
6.2 Controlled Scenarios	88
6.2.1 Goldfish	90
6.2.2 Goldfish MANRS compliant	93
6.2.3 Pram	95
6.2.4 Pram MANRS compliant	97
6.2.5 Shuttle	99
6.2.6 Shuttle MANRS compliant	103
6.3 Benchmark Scenarios	104
6.3.1 Benchmark Scenarios Evaluation	105
6.4 Summary and Discussion	112
Conclusions and Future Developments	114
Bibliography	118

List of Figures

2.1	Differences between IGP and EGP routing.	6
2.2	The Hierarchy of the Internet.	8
2.3	MANRS logo.	19
2.4	Kathará Logo.	25
2.5	Kathará architecture.	28
4.1	ROSE-T Logo.	37
4.2	Original ROSE-T architecture.	40
4.3	ROSE-T verification pipeline.	46
4.4	Example scenario compliant with Filtering Action.	49
4.5	Example scenario not compliant with Filtering Action.	49
4.6	Example scenario compliant with Anti-Spoofing Action.	51
4.7	Example scenario not compliant with Anti-Spoofing Action.	51
5.1	Topology reconstruction in old ROSE-T version.	72
5.2	Topology reconstruction in new ROSE-T version.	73
5.3	Architecture of the extended ROSE-T framework.	84
6.1	Goldfish Scenario.	90
6.2	Verification output for the Goldfish scenario.	92
6.3	Output Goldfish MANRS compliant.	94
6.4	Pram Scenario.	95
6.5	Verification output for the Pram scenario.	97

6.6	Output Pram MANRS compliant.	98
6.7	Shuttle Scenario.	100
6.8	Verification output for the Shuttle scenario.	101
6.9	Output Shuttle MANRS compliant.	104
6.10	Percentage of executed validation tests for each laboratory.	108
6.11	Comparison between the expected number of routers and the routers instantiated.	111

Chapter 1

Introduction

The Border Gateway Protocol (BGP) constitutes the backbone of inter-domain routing on the Internet, enabling thousands of independently administered Autonomous Systems (ASes) to exchange reachability information and collectively maintain global connectivity. Despite its central role, BGP was originally designed under the assumption of mutual trust between network operators, with limited built-in mechanisms to validate the correctness or legitimacy of routing announcements. This architectural characteristic has made the global routing infrastructure vulnerable to a variety of threats, including route leaks, prefix hijacking and IP address spoofing, which have repeatedly caused significant disruptions to Internet services over the past decades.

In response to these threats, the Mutually Agreed Norms for Routing Security ((**MANRS**)) initiative was established to promote a set of operational best practices aimed at improving the security and resilience of the global routing infrastructure. MANRS provides dedicated programs for different Internet stakeholders: network operators, Internet Exchange Points (IXPs), CDN and Cloud providers or equipment vendors.

This thesis focuses on the MANRS program for Network Operators, which defines four complementary actions targeting route filtering, source address validation, operational coordination and the maintenance of accurate routing registry

information.

While the initiative has gained substantial participation since its launch in 2014, verifying compliance with its requirements remains a largely manual and time-consuming process. Assessing routing security often requires inspecting complex router configurations, analysing routing policies, BGP behaviour and performing potentially disruptive validation tests on production infrastructures. As a result, operators frequently lack a safe and automated way to evaluate the impact of configuration changes before deploying them, making compliance verification both labor-intensive and error-prone.

ROSE-T is an open-source framework that supports network operators in evaluating the security of their routing infrastructure through network emulation, allowing MANRS-related checks to be performed before applying configuration changes or undergoing an official compliance assessment. Starting from the configuration of a candidate Autonomous System and a set of publicly available routing information, ROSE-T reconstructs an emulated network environment and executes a series of behavioural tests that evaluate whether the candidate network satisfies the MANRS requirements. Unlike configuration-based approaches, which analyse router configurations statically, ROSE-T observes the actual behaviour of routing policies after protocol convergence, making the verification independent of the specific vendor syntax and more representative of the operational reality.

However, at the time this work began, ROSE-T was limited to scenarios in which the candidate Autonomous System consisted of a single router. This assumption does not accurately reflect the architecture of modern networks, where an Autonomous System commonly comprises multiple interconnected border routers implementing potentially heterogeneous routing policies. As a consequence, verifying a single router provides only a partial view of the security posture of the entire AS.

This thesis addresses these limitations through four main contributions. First, **support for FRRouting (FRR)** configurations was introduced, enabling the

framework to evaluate networks built on one of the most widely adopted open-source routing platforms. Second, a mechanism for providing **local routing information** was developed, allowing the framework to operate on fully synthetic laboratory scenarios without depending on external Internet resources. Third and most significantly, the topology reconstruction process and all verification actions were redesigned to support **Multi-Router** Autonomous Systems, extending the applicability of ROSE-T to realistic deployments. Finally, **ROSET-Adapter** was developed as a companion tool that automates the extraction and preparation of all inputs required by the verification framework, enabling its application to existing Kathará laboratories with minimal manual effort.

The proposed extensions are evaluated through two complementary categories of experiments. Controlled scenarios with manually established ground truth are used to validate the correctness of the verification results, while a set of publicly available Kathará laboratories is employed to assess the robustness of the topology reconstruction pipeline across heterogeneous network configurations. The results demonstrate that the extended framework successfully detects routing policy violations and correctly validates compliant configurations across scenarios of increasing complexity, while remaining sufficiently robust to operate on laboratories that were not originally designed for MANRS verification.

The remainder of this thesis is organized as follows. Chapter 2 introduces the theoretical background on BGP, inter-domain routing relationships, routing security threats and the MANRS initiative. Chapter 3 surveys existing approaches to network verification, discussing their respective strengths and limitations. Chapter 4 describes the original ROSE-T architecture, its verification pipeline and the limitations that motivated this work. Chapter 5 presents the four contributions developed in this thesis, detailing the design and implementation choices adopted for each extension. Chapter 6 reports the experimental evaluation, including both the controlled scenarios and the benchmark experiments. Finally, the conclusions and directions for future work are discussed in Chapter "Conclusions".

Chapter 2

Preliminaries

This chapter introduces the technical background required to understand the contributions presented in the remainder of this thesis. The first section describes the Border Gateway Protocol, the de facto standard for inter-domain routing, covering its fundamental mechanisms, message types, and the business relationships that govern route propagation between Autonomous Systems. The second section analyses the main security threats affecting inter-domain routing, with particular focus on route leaks and IP spoofing, which represent the two classes of violations directly addressed by this work. The third section presents the MANRS initiative and its four operational actions, which constitute the reference framework used throughout the thesis to evaluate the routing security posture of candidate Autonomous Systems. Finally, the fourth section introduces Kathará, the network emulation platform on which ROSE-T is built, describing its architecture and the Python API that enables programmatic control of emulated network scenarios.

2.1 Border Gateway Protocol

The Border Gateway Protocol (**BGP**) is the de facto standard inter-domain routing protocol used on the Internet. Its primary purpose is to exchange reachability information between independently administered networks, known as Au-

Autonomous Systems (**ASes**)¹. Thanks to its scalability and flexibility, BGP has become one of the fundamental building blocks of the modern Internet, enabling communication between thousands of organizations, Internet Service Providers (ISPs), cloud providers and enterprise networks.

Routing protocols can generally be divided into two categories: Interior Gateway Protocols (IGPs) and Exterior Gateway Protocols (EGPs). IGPs, such as OSPF and IS-IS, are designed to distribute routing information within a single Autonomous System. In contrast, BGP operates between Autonomous Systems and is responsible for exchanging routing information across administrative boundaries. Although BGP can also be used within an AS, a distinction is usually made between External BGP (eBGP), which operates between different Autonomous Systems and Internal BGP (iBGP), which distributes externally learned routes within the same Autonomous System.

The basic principle of BGP is that neighbour routers establish a session and exchange information about the IP prefixes they can reach. Each router advertises routes learned from its own network or from other neighbours, allowing routing information to propagate across multiple Autonomous Systems. When a router receives multiple routes to the same destination, it applies a decision process based on routing attributes and local policies to determine the preferred path.

Unlike traditional shortest-path routing protocols, BGP is a path-vector protocol. Each route advertisement contains not only the destination prefix but also a set of attributes describing the path that should be followed to reach that destination. Among these attributes, the AS-PATH is particularly important, as it records the sequence of Autonomous Systems traversed by the route. This information helps prevent routing loops and allows network operators to implement complex routing policies.

Figure 2.1 illustrates the distinction between intra-domain routing, typically

¹An Autonomous System (AS) is a set of IP prefixes and routers under the control of a single administrative entity that presents a common routing policy to other Autonomous Systems. Each AS is identified by a unique Autonomous System Number (ASN).

handled by IGPs and inter-domain routing, which relies on BGP to exchange reachability information between Autonomous Systems.

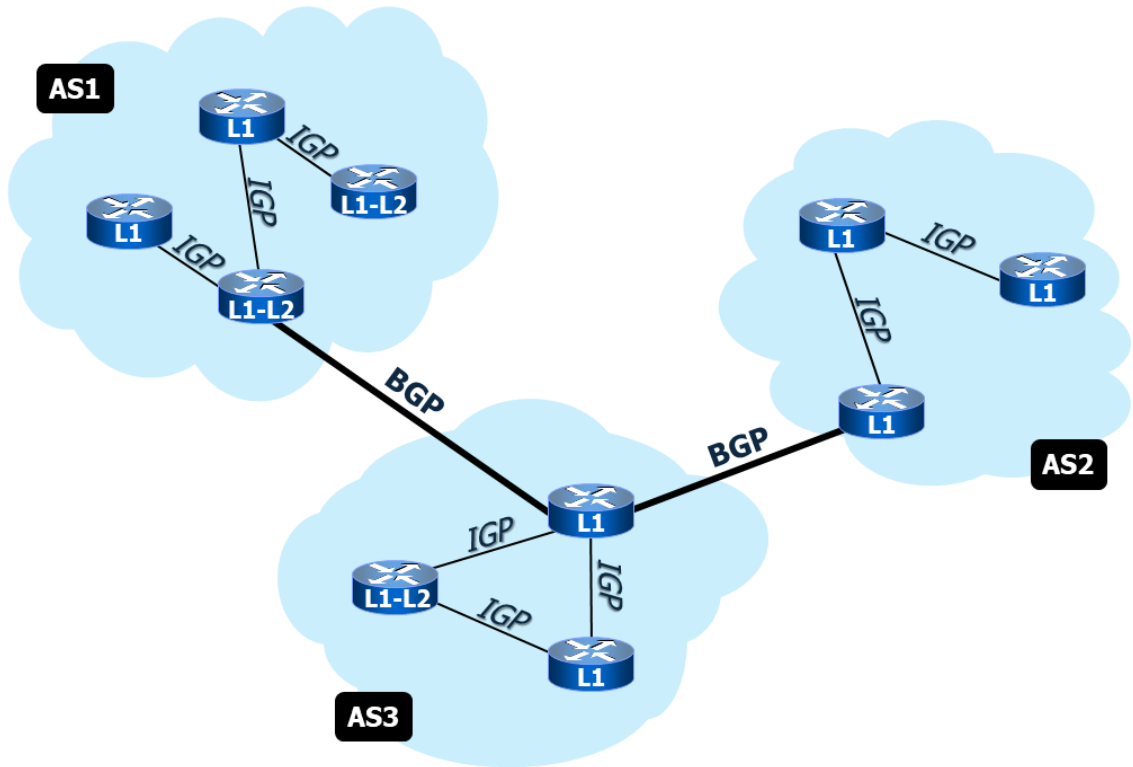


Figure 2.1: Differences between IGP and EGP routing.

BGP sessions are established over TCP, specifically using port 179. The use of TCP provides reliable transport, ensuring ordered delivery of routing information and reducing the complexity of the protocol itself. Once a session has been established, peers exchange routing information and continuously update each other whenever changes occur in network reachability.

The communication between BGP peers is performed through four main message types:

1. **OPEN**: initiates a BGP session and negotiates protocol parameters between peers.

2. **KEEPALIVE**: periodically confirms that the session is still active and operational.
3. **UPDATE**: advertises new routes, modifies existing routes, or withdraws routes that are no longer reachable.
4. **NOTIFICATION**: communicates errors and is typically used to terminate a session when a protocol violation or configuration issue is detected.

The flexibility of BGP has made it the dominant routing protocol for inter-domain communication. However, this flexibility also introduces significant operational and security challenges. Because route advertisements are largely based on trust between Autonomous Systems, incorrect configurations or malicious announcements can affect large portions of the Internet. Issues such as *route leaks*, *prefix hijacking* and *IP spoofing*, described in the next chapters, have motivated the development of operational best practices and security initiatives such as **MANRS**, view Section 2.3, which aim to improve the resilience and reliability of global routing infrastructures.

2.1.1 Inter-domain Relationships

The Internet is not a centrally managed network but rather a federation of thousands of independently administered Autonomous Systems (ASes). While BGP provides the technical mechanism for exchanging routing information, the actual propagation of routes is largely determined by the economic and business relationships established between network operators.

The most common relationship is the *customer-provider* relationship. In this model, the customer pays the provider to obtain connectivity to the rest of the Internet. As a consequence, providers typically advertise all routes they know to their customers, while customers advertise their own routes and those of their downstream customers to their providers.

A second common relationship is *settlement-free peering*. In this case, two Autonomous Systems agree to exchange traffic between their respective customers without financial compensation. Peering allows networks to reduce transit costs and improve performance by exchanging traffic directly rather than through an upstream provider.

These relationships create a hierarchical structure often represented as a graph, where large transit providers occupy higher levels and smaller networks appear lower in the hierarchy. Figure 2.2 illustrates the most common types of inter-AS relationships.

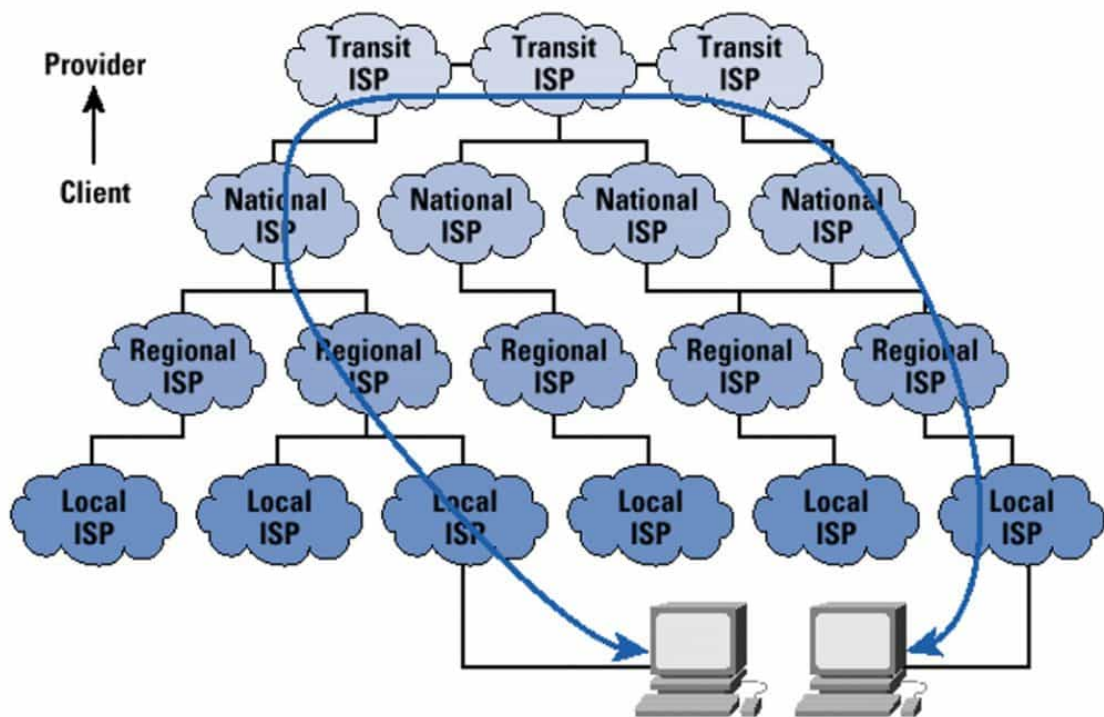


Figure 2.2: The Hierarchy of the Internet.

From a routing perspective, these business agreements directly influence the propagation of BGP announcements. Network operators generally follow policies that maximize revenue and minimize transit costs. As a result, routes learned

from customers are usually preferred over routes learned from peers or providers, since customer traffic generates revenue while provider traffic incurs costs.

The routing policies derived from these economic relationships give rise to the so-called **valley-free** routing model. According to this model, a valid AS path may consist of a sequence of customer-to-provider links, optionally followed by a single peer-to-peer link and then a sequence of provider-to-customer links. Intuitively, traffic is expected to move "up" the hierarchy toward transit providers, possibly cross through a peering relationship and then move "down" toward the destination network.

Paths that violate this pattern are called *valley paths*. For example, if an Autonomous System were to advertise to one provider a route learned from another provider, it would effectively offer free transit services, contradicting the underlying business agreements. Consequently, such announcements are typically filtered by routing policies and are rarely observed in normal Internet operation.

The valley-free model plays a fundamental role in understanding Internet routing behaviour, route propagation and the structure of the global routing system. Furthermore, knowledge of inter-AS relationships is essential when analysing routing security and operational best practices. Several security mechanisms and validation frameworks rely on the accurate representation of routing policies and business relationships between Autonomous Systems.

2.1.2 Internet Routing Registries

Internet Routing Registries (**IRRs**) are publicly accessible databases used by network operators to publish routing information and routing policies associated with their Autonomous Systems. Their primary purpose is to facilitate routing coordination between network operators, improve route filtering practices and provide a standardized way to document routing policies and resource ownership.

Unlike routing protocols such as BGP, IRRs do not participate directly in route propagation. Instead, they act as repositories of declarative information

that can be consulted by operators and automated tools. The information stored in IRRs can be used to generate routing filters, validate routing policies and support operational activities involving multiple Autonomous Systems.

Several organizations maintain IRR databases, including regional Internet registries and independent operators. Historically, these registries have represented one of the primary sources of information regarding routing policies and Autonomous System relationships on the Internet.

IRR information is organized into objects, each describing a specific aspect of routing operations. Among the most important object types are:

- **route objects**, which associate an IP prefix with the Autonomous System authorized to originate it;
- **aut-num objects**, which describe an Autonomous System and its routing policies, including route import and export rules;
- **as-set objects**, which group multiple Autonomous Systems into a logical collection;
- **route-set objects**, which define collections of IP prefixes.

Routing policies within IRRs are commonly expressed using the Routing Policy Specification Language (RPSL), a language specifically designed to describe inter-domain routing relationships and policy constraints. Through RPSL, operators can specify which routes are accepted from neighbouring Autonomous Systems and which routes are advertised to them.

One of the most common applications of IRR data is the automated generation of route filters. By consulting route and policy information published by a customer, a provider can automatically construct filtering rules that accept only authorized route announcements. This reduces the risk of accidental route leaks and routing misconfiguration.

Despite their widespread adoption, IRRs present several limitations. The information contained within them is generally maintained manually and may become outdated, incomplete, or inconsistent with the actual operational state of the network. As a result, the routing policies and resource assignments described in an IRR do not necessarily reflect the behaviour currently observed on the Internet. For this reason, IRR data should be considered a valuable operational resource, but not an authoritative guarantee of routing correctness.

2.2 Routing Security Threats

The Internet is a highly decentralized system composed of thousands of independently managed Autonomous Systems that continuously exchange routing information through BGP. While this distributed architecture provides remarkable scalability and resilience, it also introduces significant security challenges. Since BGP was originally designed under the assumption of mutual trust between network operators, many routing decisions are still based on information received from neighbouring Autonomous Systems without intrinsic mechanisms to verify its correctness or legitimacy.

As a consequence, both accidental misconfiguration and malicious actions can propagate beyond the boundaries of a single network, potentially affecting Internet connectivity on a global scale. Incorrect routing announcements may divert traffic through unintended paths, disrupt communication between networks, or expose traffic to unauthorized intermediate Autonomous Systems. Similarly, the absence of source address validation allows attackers to forge packet source addresses, enabling a variety of attacks that are difficult to trace back to their origin.

Over the years, numerous threats targeting inter-domain routing have been identified, including prefix hijacking, route leaks, IP spoofing, denial-of-service attacks and routing policy violations. While each of these issues presents differ-

ent causes and consequences, they all stem from the lack of built-in validation mechanisms within the original Internet routing architecture.

Among these threats, this work focuses on two representative classes that are directly addressed by the MANRS initiative and supported by the verification framework presented in this thesis. The first concerns *route leaks*, which arise when routing information is propagated in violation of established inter-domain routing policies. The second concerns *IP spoofing*, whereby forged source addresses are used to disguise the origin of network traffic or amplify distributed attacks. These two threats exemplify different aspects of routing security: one affects the control plane by altering the propagation of routing information, while the other targets the data plane by manipulating packet forwarding behaviour.

The following sections provide a detailed discussion of both threats, illustrating their causes, operational impact and the mitigation techniques adopted in modern operational networks.

2.2.1 Route leaks

Inter-domain routing incidents can generally be classified into two broad categories according to the aspect of the routing process they affect. The first category includes attacks targeting the legitimacy of route origination, commonly referred to as *prefix hijacking*. The second category comprises violations of routing propagation policies, known as *route leaks*. Although both phenomena may lead to traffic redirection, service disruption, or large-scale Internet outages, they originate from different causes and therefore require different mitigation mechanisms.

A prefix hijack occurs when an Autonomous System advertises IP prefixes that it is not authorized to originate. As a consequence, other networks may incorrectly select the offending AS as the best destination for those prefixes, diverting traffic away from its legitimate owner. Prefix hijacking therefore concerns the authenticity of routing information and is effectively mitigated through mech-

anisms such as the Resource Public Key Infrastructure (RPKI) and Route Origin Validation (ROV) introduced in [MSW⁺13].

Route leaks, instead, do not necessarily involve incorrect prefix ownership. In most cases, the advertised prefixes legitimately belong to the originating Autonomous System, but they are propagated beyond their intended scope due to violations of inter-domain routing policies. Rather than questioning who owns a prefix, according to [SMM⁺16], route leaks concern how routing information is propagated across Autonomous Systems. Table 2.1 summarizes the main differences between prefix hijacking and route leaks. Since the focus of this thesis is the verification of routing policies according to the MANRS recommendations, the remainder of this section concentrates on route leaks.

Characteristic	Prefix Hijacking	Route Leak
Primary issue	Unauthorized advertisement of a prefix	Unauthorized propagation of a legitimate route
Affected property	Route origin authenticity	Routing policy enforcement
Origin AS	Incorrect or forged	Legitimate
Main cause	Malicious advertisement	Misconfiguration
Typical consequence	Traffic redirected to the wrong origin	Traffic propagated through unintended transit paths
Primary mitigation	RPKI and Route Origin Validation (ROV)	Route filtering, routing policies, IRRs

Table 2.1: Comparison between prefix hijacking and route leaks.

Main Causes The majority of route leaks originate from configuration errors rather than malicious activity. Incorrect route filtering, missing export policies, or improperly configured route maps may cause an Autonomous System to advertise routes learned from one external neighbour to another neighbour that should not receive them.

A common example involves a customer Autonomous System that advertises routes learned from one provider to a second provider, effectively offering unauthorized transit services. According to the valley-free routing model introduced in the previous section, routes learned from providers or peers should generally only be advertised to customers. Violating this principle creates routing paths that conflict with the intended commercial relationships between Autonomous Systems and may unintentionally redirect large volumes of Internet traffic.

Although accidental misconfigurations remain the predominant cause, route leaks may also be intentionally introduced to manipulate traffic engineering decisions or bypass routing restrictions. However, such cases are considerably less common than operational mistakes.

Potential Impact The consequences of a route leak largely depend on the size and connectivity of the involved Autonomous Systems. In limited scenarios, the impact may be confined to a small number of affected networks. However, when the leaked routes are propagated by highly connected providers, the consequences can rapidly extend to a significant portion of the global Internet.

Traffic may be diverted through unexpected paths, resulting in increased latency, congestion, packet loss, or complete loss of connectivity. Furthermore, the leaked Autonomous System may become an unintended transit provider, receiving traffic volumes far beyond its operational capacity. Several large-scale Internet outages have been attributed to route leaks that propagated globally within minutes due to the distributed nature of BGP.

Besides performance degradation, route leaks also introduce security concerns. Redirecting traffic through unintended networks may expose communications to additional intermediate Autonomous Systems, increasing the risk of traffic inspection or facilitating subsequent attacks against the routing infrastructure.

Mitigation Techniques Preventing route leaks primarily relies on the correct implementation of routing policies at the borders of each Autonomous System.

Export filters should ensure that only routes consistent with the intended business relationships are propagated to neighbouring networks. Similarly, import policies should restrict the acceptance of routes according to the role of each external peer.

Internet Routing Registries (IRRs) provide an important mechanism for documenting routing policies and inter-domain relationships. By publishing routing policies using the Routing Policy Specification Language (RPSL), network operators enable both manual and automated validation of their routing configurations against the declared policies.

Additional protection is provided by Route Origin Validation (ROV), based on the Resource Public Key Infrastructure (RPKI). Although RPKI does not directly prevent route leaks, it mitigates a related class of attacks by verifying that the origin Autonomous System is authorized to announce a given prefix, as mentioned earlier when discussing hijacking. Consequently, RPKI and routing policy validation should be regarded as complementary mechanisms rather than alternative solutions.

Finally, initiatives such as MANRS promote operational best practices that combine accurate routing policies, filtering mechanisms, routing registry maintenance and continuous monitoring to reduce the likelihood of route leaks and improve the overall security of inter-domain routing.

2.2.2 Spoofed Packets

The Internet Protocol was designed with scalability and interoperability as primary objectives, rather than strong security guarantees. As a consequence, the source IP address contained in an IP packet is not intrinsically authenticated, allowing a sender to forge arbitrary source addresses before transmitting traffic. Packets generated in this way are commonly referred to as *spoofed packets*.

IP spoofing has been exploited in a wide variety of attacks throughout the history of the Internet. Examples include denial-of-service attacks, reflection and

amplification attacks exploiting publicly accessible services, attempts to conceal the identity of an attacker and the circumvention of filtering mechanisms based on source addresses. The impact and techniques associated with spoofing have therefore been extensively studied, leading to several mitigation strategies proposed by both the research community and the Internet Engineering Task Force (IETF).

From the perspective of network security, however, the responsibility for mitigating IP spoofing does not lie exclusively with the destination network. A fundamental principle promoted by the IETF is that each Autonomous System should prevent the generation of spoofed traffic originating from its own infrastructure. This concept, formalized by the Network Ingress Filtering recommendations [SF00] and later extended to multi-homed environments [BS04], shifts the focus from detecting spoofed packets to preventing their propagation across the Internet. Since this principle is also one of the core recommendations promoted by MANRS, the remainder of this section concentrates on the mechanisms available to network operators for preventing the transmission of spoofed traffic rather than on the attacks that exploit it.

Main Causes The primary cause of IP spoofing is the absence of source address validation at the edge of a network. If border routers forward packets without verifying whether the source IP address legitimately belongs to the originating network, malicious or compromised hosts can transmit packets using arbitrary source addresses.

In operational networks, this situation is generally the consequence of missing or incorrectly configured ingress filtering policies rather than protocol vulnerabilities. While modern routing infrastructures provide several mechanisms for validating packet origins, these mechanisms must be explicitly enabled and correctly configured by network operators. The problem becomes even more challenging in multi-homed environments, where multiple valid forwarding paths may exist

simultaneously and traditional validation techniques require additional considerations to avoid discarding legitimate traffic.

Potential Impact Allowing spoofed packets to leave an Autonomous System has consequences that extend well beyond the originating network. Attackers may conceal their identity by forging unreachable or unrelated source addresses, significantly complicating traceback and incident response activities.

More importantly, spoofed packets enable large-scale reflection and amplification attacks. By sending requests with a forged victim address to publicly accessible services, an attacker can cause multiple third-party systems to direct their responses towards the victim, multiplying the attack traffic without exposing the true source of the attack. Such techniques have repeatedly been employed in distributed denial-of-service (DDoS) campaigns targeting Internet services.

Although the specific attack techniques continue to evolve, they all rely on the same fundamental assumption: networks allow the transmission of packets whose source addresses have not been validated. Consequently, preventing spoofed traffic at its point of origin significantly reduces the attack surface available to malicious actors.

Mitigation Techniques The most widely adopted mitigation best practice is *Network Ingress Filtering*, introduced in BCP38 and later refined by BCP84 to address the additional complexity of multi-homed Autonomous Systems. The underlying principle is simple: border routers should discard outgoing packets whose source addresses are inconsistent with the prefixes legitimately reachable through the interface on which they were received.

Several implementation techniques can be used to enforce this principle. Static access control lists provide a simple but relatively inflexible solution, while routing-aware mechanisms such as Unicast Reverse Path Forwarding (uRPF), described in [SMH20], automatically validate packet origins using the router's forwarding table. Depending on the network topology, uRPF can operate in different modes,

including strict and loose validation, allowing operators to balance security and operational flexibility according to their routing architecture.

In addition to these filtering mechanisms, operational initiatives such as MANRS encourage network operators to systematically deploy source address validation as part of a broader set of routing security best practices. Rather than introducing new protocols, these recommendations promote the consistent application of well-established operational techniques that significantly reduce the ability of attackers to exploit IP spoofing on the public Internet.

2.3 MANRS

"Mutually Agreed Norms for Routing Security (MANRS) is a global initiative, supported by the Global Cyber Alliance, that provides crucial fixes to reduce the most common routing threats. MANRS offers specific actions via four programs for Network Operators, Internet Exchange Points, CDN and Cloud Providers and Equipment Vendors." [MAN]



Figure 2.3: MANRS logo.

The increasing dependence of modern society on Internet connectivity has made the resilience and security of the global routing infrastructure a matter of strategic importance. Although several technologies have been proposed to improve the security of inter-domain routing, many incidents are still caused by operational misconfiguration rather than protocol vulnerabilities. Consequently,

improving Internet routing security requires not only new technical mechanisms but also the widespread adoption of common operational best practices.

To address this need, the Internet Society launched the Mutually Agreed Norms for Routing Security initiative (MANRS), whose logo is shown in the Figure 2.3, in 2014. Rather than defining new routing protocols, MANRS promotes a collection of operational best practices aimed at reducing the most common routing threats through collaborative action among internet stakeholders. Participation in the initiative is voluntary, but organizations joining MANRS publicly commit to implementing a set of security recommendations and to continuously maintaining their routing infrastructure according to these principles.

The initiative was motivated by the observation that many large-scale routing incidents could have been prevented through relatively simple operational measures, such as proper route filtering, source address validation, accurate routing registry maintenance and effective coordination between network operators. Rather than attempting to eliminate every possible routing attack, MANRS focuses on reducing the most frequent causes of routing incidents by promoting practices that are already technically feasible and widely supported by existing routing software.

Since its introduction, MANRS has progressively expanded its scope beyond traditional network operators, at the time of writing, according to the official website, MANRS counts 1,354 participants worldwide, at the time of writing, MANRS counts 1,354 participants worldwide, a remarkable figure given the specialized audience the initiative targets. While the original initiative primarily targeted Autonomous System operators, the growing complexity of the Internet ecosystem highlighted the need to involve other stakeholders that play a key role in routing security. As a result, MANRS now defines dedicated programs tailored to four categories of participants:

- **Network Operators:** focuses on the secure operation of Autonomous Sys-

tems by promoting best practices for route filtering, source address validation, routing coordination, and routing information accuracy.

- **Internet Exchange Points (IXPs):** provides recommendations for the secure operation of route servers and exchange infrastructures, encouraging routing security mechanisms and operational best practices within Internet exchange fabrics.
- **CDN and Cloud Providers:** addresses the specific challenges faced by large-scale service providers, promoting secure routing practices for customer prefixes and globally distributed infrastructures.
- **Equipment Vendors:** encourages router and networking equipment manufacturers to integrate routing security features into their products, simplifying the adoption of MANRS recommendations by network operators.

Although all four programs pursue the common objective of improving the security and resilience of the global routing infrastructure, each defines requirements tailored to the responsibilities of the corresponding stakeholder.

The remainder of this section focuses on the MANRS program for **Network Operators**, as it provides the reference framework adopted throughout this thesis and constitutes the basis of the verification process implemented by ROSE-T. Consequently, unless otherwise stated, the term *MANRS* will refer to the Network Operators program throughout the remainder of this document.

The MANRS program for Network Operators is structured around four complementary actions, each targeting a different aspect of inter-domain routing security. Although each action focuses on a specific operational domain, they should not be regarded as independent requirements. Instead, they form a comprehensive framework in which every action contributes to improving the overall resilience and security of the global routing ecosystem.

The four actions, however, do not all have the same role within the MANRS compliance process. Actions 1 (Filtering), 3 (Coordination) and 4 (Global Vali-

dation) constitute mandatory requirements for participation in the MANRS Network Operators program. Action 2 (Anti-Spoofing) differs slightly: rather than requiring operators to demonstrate a specific implementation technique, MANRS requires participants to periodically execute source address validation tests using the CAIDA Spoofer project² and to make the results publicly available. Consequently, compliance is based on demonstrating that anti-spoofing verification has been performed, rather than on the outcome of the tests themselves.

The following sections briefly introduce each MANRS action for Network Operators and summarize the corresponding operational recommendations.

2.3.1 Action 1 – Filtering

The first MANRS action aims to prevent the propagation of incorrect routing information by ensuring that routing announcements comply with the routing policies established between neighbouring Autonomous Systems. In practice, operators are expected to configure appropriate import and export filters so that routes learned from one external neighbour are only propagated to other neighbours when permitted by the underlying business relationships.

This action primarily addresses routing policy violations such as route leaks, which, as discussed in the previous chapter, generally originate from configuration errors rather than malicious behaviour. Proper route filtering prevents an Autonomous System from unintentionally providing transit services or advertising routes beyond their intended scope, thereby reducing the likelihood of large-scale routing incidents.

The specific implementation is left to the network operator and may involve route maps, prefix filters, AS-path filtering, or other policy enforcement mech-

²CAIDA (Center for Applied Internet Data Analysis) is a research center at the University of California, San Diego, that develops tools and datasets for the analysis of the Internet infrastructure. Among its services, CAIDA maintains the *Spoofer* project, a distributed measurement platform used to assess source address validation deployment by collecting the results of anti-spoofing tests performed by network operators [BB05].

anisms supported by the routing platform. Regardless of the implementation details, the objective remains the same: ensure that only authorized routing information is propagated to external neighbours.

This action refers to the discussion in Section 2.2.1.

2.3.2 Action 2 – Anti-Spoofing

The second action focuses on preventing the transmission of packets with spoofed source IP addresses. In accordance with the recommendations defined by BCP 38 and BCP 84, participating Autonomous Systems should validate the source addresses of packets entering their networks and discard traffic whose origin is inconsistent with the expected forwarding paths.

Several implementation techniques are available, including static ingress filtering and Unicast Reverse Path Forwarding (uRPF), which automatically verifies source address reachability using the router forwarding tables. Depending on the network topology, different validation modes may be adopted to balance operational flexibility and security requirements.

By preventing spoofed traffic from leaving individual networks, this action contributes to reducing the effectiveness of reflection attacks, distributed denial-of-service campaigns and other attacks that rely on forged source addresses.

Unlike the other MANRS actions, compliance with this requirement is not evaluated through the adoption of a specific anti-spoofing mechanism. Instead, the MANRS program requires participants to execute periodic tests using the CAIDA Spoofer software on publicly reachable network segments and to publish the results through the CAIDA Spoofer database. The objective is therefore to demonstrate that source address validation is regularly assessed, independently of the particular implementation adopted within the network.

A more detailed analysis on what is a spoofed packet and the threats linked to this, can be found in Section 2.2.2.

2.3.3 Action 3 – Coordination

The third action addresses the operational aspect of routing security by promoting effective communication between network operators. Routing incidents often require rapid interaction between the organizations involved in order to identify the source of the problem and deploy corrective actions before the disruption propagates further.

For this reason, MANRS recommends maintaining accurate and publicly accessible operational contact information, including Network Operations Center (NOC) ³ and abuse contacts. Keeping these contacts up to date allows operators to coordinate incident response efficiently and significantly reduces the time required to resolve routing problems affecting multiple Autonomous Systems.

Although this action does not directly modify routing behaviour, it represents an essential component of operational security by facilitating collaboration within the Internet community.

2.3.4 Action 4 – Global Validation

The fourth action promotes the publication and maintenance of accurate routing information in globally accessible repositories. Specifically, network operators are encouraged to maintain consistent records within Internet Routing Registries (IRRs) and, where possible, deploy Resource Public Key Infrastructure (RPKI) to enable external validation of routing information.

Publishing accurate routing policies and prefix ownership information allows other operators to automatically verify routing announcements, build reliable filtering policies and detect inconsistencies between publicly declared information and observed routing behaviour.

³Network Operations Center: In computer networking and IT, a Network Operations Center is a centralized facility where IT professionals monitor, manage and maintain network infrastructure to ensure optimal performance, security and uptime. Key functions include network monitoring, incident response and troubleshooting.

This action complements the previous three by improving the transparency of inter-domain routing. Rather than preventing routing incidents directly, it provides the information required for external validation and facilitates the deployment of automated routing security mechanisms across the Internet.

Action	Primary Objective
Action 1 – Filtering	Route filtering, policies, route leak prevention
Action 2 – Anti-Spoofing	Ingress filtering, uRPF, source address validation
Action 3 – Coordination	NOC contacts, abuse contacts, incident response
Action 4 – Global Validation	IRR records, RPKI, information consistency

Table 2.2: Summary of the four MANRS actions.

Table 2.2 summarizes the primary objective of each MANRS action. Together, these four recommendations constitute the reference framework used throughout the remainder of this thesis to evaluate the routing security posture of candidate Autonomous Systems.

2.4 Kathará

"Kathará is a network emulation system which accurately reproduces the behaviour of a real system. It can exploit several virtualization technologies leveraging on its modularity."[SAC20]



Figure 2.4: Kathará Logo.

The analysis, validation and experimentation of routing protocols require an environment in which complex network topologies can be reproduced safely, rapidly and repeatedly. Performing such activities directly on production infrastructures is generally impractical due to the potential impact on operational services, while purely theoretical analyses often fail to capture the dynamic behaviour of routing protocols during message exchanges, convergence and policy enforcement.

For these reasons, network emulation has become a widely adopted approach in both networking research and education. Unlike network simulators, which model the behaviour of protocols through abstractions, network emulators execute the same software and protocols used in real deployments, providing a faithful representation of the actual network behaviour. This allows researchers and network operators to evaluate routing policies, validate configurations, reproduce network incidents and perform controlled experiments without affecting operational infrastructures.

Among the available network emulation platforms, Kathará, whose logo is visible in Figure 2.4, has established itself as a lightweight and flexible framework specifically designed for networking experiments. Originally presented in [BILDB18] and continuously maintained as an open-source project, Kathará enables the creation of realistic network laboratories while requiring only modest computational resources.

The entire architecture of Kathará is based on two fundamental concepts:

- **Devices:** virtual network elements that emulate real networking equipment. A device may represent a router, host, firewall, DNS server, or virtually any other network appliance. Each device executes its own operating system and networking software, making its behaviour equivalent to that of a physical machine;
- **Collision Domains:** virtual Layer-2 broadcast networks interconnecting

the interfaces of multiple devices. From the perspective of the emulated machines, collision domains behave as standard Ethernet segments, transparently forwarding packets between connected interfaces.

The combination of devices and collision domains forms a complete network laboratory, referred to as a *network scenario*. Each laboratory is described by a directory containing all the resources required to instantiate the emulated infrastructure. The most relevant files are:

- **lab.conf**, which describes the network topology by defining the participating devices and the collision domains connecting them;
- **Startup scripts**, executed when each virtual device boots, typically used to configure interfaces, IP addresses, static routes and other operating system parameters;
- **Device-specific configuration files**, containing the configuration of networking services executed by each device, such as routing daemons (e.g., FRRouting), DNS servers, or other applications.

Kathará follows a modular architecture, illustrated in Figure 2.5, composed of three main components.

- The **CLI** provides the user interface used to create, deploy, inspect and remove network laboratories.
- The **Parser** interprets the laboratory description and generates an internal representation of the network topology.
- The **Manager** interfaces with the selected virtualization backend, translating the abstract laboratory description into the corresponding virtual resources required to instantiate the emulated network.

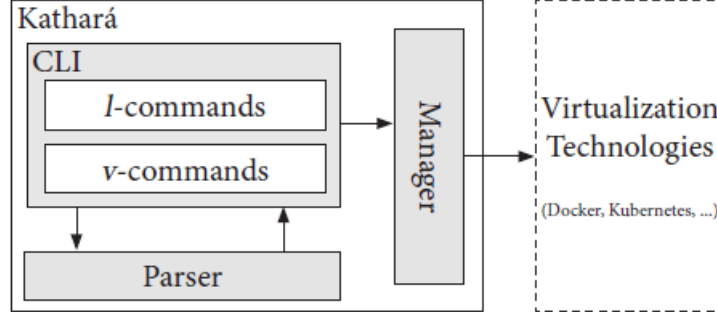


Figure 2.5: Kathará architecture.

One of the main strengths of Kathará is its modularity. Rather than implementing its own virtualization engine, it relies on external technologies such as Docker [Mer14] to instantiate virtual devices. This design allows the same laboratory description to remain independent of the underlying virtualization platform while benefiting from lightweight container-based execution.

The adoption of container virtualization significantly reduces the computational overhead typically associated with traditional virtual machines, enabling the deployment of complex network scenarios on commodity hardware. Furthermore, this design naturally supports horizontal scalability. As demonstrated by the *Megalos* architecture, Kathará can transparently distribute virtual devices across multiple physical hosts managed by *Kubernetes*⁴ while preserving the abstraction of a single network laboratory. By combining lightweight containers with distributed orchestration and efficient virtual networking, scenarios comprising thousands of virtual routers and Layer-2 domains can be emulated with limited overhead, making the framework suitable not only for small educational laboratories but also for large-scale research and experimentation environments, further details on *Megalos* are provided in [SADBP21].

⁴Kubernetes (K8s) is an open-source container orchestration platform originally developed by Google and currently maintained by the Cloud Native Computing Foundation (CNCF). It automates the deployment, scaling and management of containerized applications [BGO⁺16].

2.4.1 Kathará Python API

Although Kathará can be operated through its command-line interface, one of its most powerful features is the availability of a comprehensive Python API that exposes virtually every aspect of the network emulation framework. Rather than manually creating laboratory files and interacting with the emulator through shell commands, developers can programmatically construct, configure, deploy and control complete network scenarios directly from Python applications.

This capability is fundamental for the development of automation frameworks such as ROSE-T. Instead of treating network emulation as a manual process, the Python API allows the entire lifecycle of an experiment to be integrated into a software pipeline. Network topologies can be generated dynamically, router configurations can be produced automatically, laboratories can be instantiated on demand and the execution of verification procedures can be coordinated without any manual intervention.

The API provides high-level abstractions for all the entities composing a Kathará laboratory. Developers can create virtual machines, connect them through collision domains, generate startup scripts, assign configuration files, start and stop individual devices, execute commands inside running containers and retrieve their output programmatically. As a result, the emulated network becomes an integral part of the application rather than an external component controlled through scripts.

Another significant advantage of this approach is reproducibility. Since laboratories are entirely described through software, the same scenario can be generated repeatedly under identical conditions, enabling deterministic experiments and facilitating both debugging and large-scale automated testing. Furthermore, because the laboratory description is produced dynamically, it can be adapted according to the input data provided by the user without requiring pre-built templates for every possible network topology.

These characteristics make the Kathará Python API considerably more than a

convenience layer over the command-line interface. They transform the emulator into a programmable network experimentation platform capable of supporting advanced applications in network verification, protocol testing, cybersecurity research and education. In the context of this thesis, the API constitutes the enabling technology that allows ROSE-T to automatically reconstruct, deploy, verify and dismantle complete network scenarios as part of a fully automated MANRS compliance verification workflow.

Chapter 3

State of the Art

Verifying the behaviour of a router or an Autonomous System (AS) is not a new problem. Over the years, numerous tools and methodologies have been developed to analyse, validate and troubleshoot routing configurations and network behaviour with varying levels of accuracy and scalability. These solutions aim to ensure that routing policies are correctly implemented, that reachability requirements are satisfied and that the network behaves as expected under different operating conditions.

Existing approaches can generally be grouped into a few categories, each characterized by different assumptions, verification techniques and operational trade-off. Some tools focus on analysing control-plane information and routing configurations to infer network behaviour, while others rely on active measurements or data-plane observations to validate the actual forwarding paths taken by packets.

Although these approaches share the common goal of improving network reliability and correctness, they differ significantly in terms of visibility, computational complexity, scalability and the guarantees they can provide. Understanding these differences is essential for evaluating the strengths and limitations of current solutions and for identifying the challenges that remain unsolved.

In the following sections, we examine the two main approaches adopted by

existing verification systems, highlighting their underlying principles, similarities, key differences and the critical issues that affect their effectiveness in real-world deployments.

3.1 Configuration Verifiers

The first and historically most common approach to network verification is based on the analysis of routing configurations. Rather than observing the actual behaviour of the network, these systems inspect the configuration files deployed on routers and reconstruct the expected network state by modelling the effects of routing protocols, routing policies and forwarding rules.

The fundamental assumption behind this approach is that the network configuration represents the source of truth. By parsing router configurations, a verification system can infer routing relationships, route propagation constraints, policy decisions and forwarding behaviour. This enables operators to verify properties such as reachability, path preferences, routing loops, black holes, route filtering policies and network isolation requirements before changes are deployed to production.

One of the main advantages of configuration-based verification is its ability to perform proactive analysis. Since it does not require access to a running network, verification can be integrated into network design and deployment workflows, allowing engineers to identify potential issues before they affect production traffic. Furthermore, the approach scales well to large networks and supports extensive "what-if" analyses that would be difficult or risky to perform on live infrastructures.

However, the effectiveness of this method is inherently limited by the information contained in the configurations themselves. Modern networks exhibit a variety of dynamic behaviours that cannot always be captured through static configuration analysis alone. For example, failures of links or devices, protocol

convergence dynamics, transient routing states, software bugs, hardware malfunctions, route flap damping effects and traffic-engineering decisions based on real-time measurements may significantly influence network behaviour despite not being explicitly represented in configuration files.

Similar challenges arise when considering external dependencies. An Autonomous System may receive routes from neighbouring networks whose policies are unknown or subject to change over time. While a configuration-based verifier can model the local routing policies applied to these routes, it generally cannot predict how external networks will advertise, filter, or modify routing information. As a result, the computed network state may differ from the one observed in operation.

Consider, for instance, a router configured to prefer a particular BGP route according to its local preference policy. A configuration analysis tool may correctly conclude that the route will be selected under normal conditions. However, if the preferred route becomes unavailable due to an upstream failure, if a neighbouring AS changes its export policy, or if routing information is modified by mechanisms not reflected in the local configuration, the actual forwarding behaviour may deviate from the predicted one.

Despite these limitations, configuration-based verification remains one of the most widely adopted approaches in both academia and industry. Several tools implement this methodology, including frameworks such as *Batfish*, which analyse network configurations to infer control-plane and data-plane behaviour and verify network-wide properties. A detailed discussion of specific tools is outside the scope of this section and can be found in their respective documentation and related literature [FFP⁺15].

3.2 Real-time Monitoring

The second major approach to network verification is based on the observation of the operational network state. Rather than inferring behaviour from router configurations, these systems collect information directly from running devices and routing protocols in order to verify the properties of the deployed network.

Typical sources of information include routing tables, forwarding tables, link-state databases, BGP updates, protocol telemetry and active measurements. By analysing the state currently maintained by routers, these systems can verify reachability, routing consistency, route propagation and other operational properties.

Compared to configuration-based verification, this approach provides a more accurate view of the network as it actually operates. Configuration deployment errors, software defects, hardware failures and unexpected protocol interactions can often be detected because they manifest themselves in the observed operational state, even when they are not apparent from the configuration files.

However, operational-state verification is inherently reactive. Since the analysis is performed on the running network, it can only verify conditions that are currently observable. While this makes it effective for monitoring and troubleshooting, it provides limited support for evaluating hypothetical scenarios or predicting the impact of future configuration changes. Furthermore, obtaining a complete view of the network may require access to operational data that is not always available, particularly when external Autonomous Systems are involved.

Several systems and data collection infrastructures follow this philosophy, including route collectors, routing telemetry platforms and monitoring services that continuously gather information from operational networks. These tools play a fundamental role in understanding the current behaviour of Internet routing, although they generally do not provide the ability to experimentally evaluate new scenarios before deployment.

Several systems and infrastructures adopt this philosophy by continuously collecting and analysing routing information from operational networks. Examples include *RouteViews* [Mey03], which gathers BGP routing data from geographically distributed collectors and *ARTEMIS* [SKG⁺18], a framework that leverages real-time routing information to detect and mitigate BGP hijacking events. These systems provide valuable insights into the actual state of Internet routing, enabling the detection of anomalies and policy violations that may not be evident through static configuration analysis alone.

3.3 Emulation and Testing

Both approaches present significant limitations when the goal is to assess compliance with operational best practices. Configuration-based systems can verify the presence of routing policies but cannot always demonstrate their effectiveness, while operational-state systems are limited to observing the current behaviour of the deployed network from an external point of view. This motivates a third approach based on network emulation and controlled experimentation, which is the foundation of **ROSE-T**.

Network emulation combines several advantages of both configuration-based verification and operational-state analysis. On the one hand, it aims to reproduce the behaviour of the real network with a high degree of fidelity by executing actual routing software, protocol implementations and routing policies. This allows the verification process to observe protocol interactions and forwarding decisions in a way that is often closer to operational reality than purely static analyses.

On the other hand, because the network is recreated within an isolated and controlled environment, emulation provides a level of flexibility that is difficult to achieve on production infrastructures. Network operators can safely inject routing announcements, simulate failures, evaluate policy changes and reproduce attack scenarios without affecting live traffic or disrupting network services.

This capability is particularly valuable when assessing security and operational best practices. Many properties cannot be fully validated by simply inspecting configurations, as the presence of a policy does not necessarily guarantee its correct behaviour under all operating conditions. Through emulation, it becomes possible to experimentally verify whether routing policies are actually enforced and to observe how the network reacts to specific events or malicious inputs.

As a result, emulation-based verification occupies an intermediate position between static configuration analysis and operational-state monitoring. It retains the possibility of performing controlled experiments and evaluating hypothetical scenarios, while at the same time providing a more realistic representation of network behaviour than approaches based only on configuration analysis.

Chapter 4

ROSE-T

Previous chapters introduced the theoretical concepts underlying routing security, the operational recommendations promoted by MANRS, explored in detail in Section 2.3 and the role of network emulation in the validation of routing protocols. Building upon these foundations, this chapter presents ROSE-T, an open-source framework designed to automatically verify the compliance of an Autonomous System with the MANRS recommendations. The tool was presented in 2024 and further details are provided in [MSPa] while the software code is publicly available on GitHub [MSPb].



Figure 4.1: ROSE-T Logo.

Unlike traditional configuration analysis tools, ROSE-T, whose logo is presented in Figure 4.1, adopts an active verification approach, as described in Section 3.3. Starting from the configuration of a candidate Autonomous System and a set of publicly available routing information, the framework reconstructs an emulated network environment, deploys it, allows the routing protocols to converge and executes a series of tests that evaluate the actual behaviour of the network. This methodology enables the verification process to consider the combined effects of routing policies, protocol interactions and network dynamics, rather than relying exclusively on static configuration analysis.

ROSE-T was originally developed within the Kathará Framework project, described in Section 2.4 and released as an open-source research tool to demonstrate the feasibility of automated MANRS compliance verification through network emulation. At the time this work began, the framework already provided a complete verification pipeline capable of validating several MANRS actions, but it also relied on a number of simplifying assumptions that limited its applicability to relatively simple network scenarios.

It is important to emphasize that ROSE-T is not intended to automatically certify that an Autonomous System is MANRS compliant. The official MANRS validation process relies on a broader set of procedures, data sources and operational assessments that extend beyond the scope of the framework. Consequently, a successful execution of all ROSE-T verification actions should not be interpreted as a formal certification of compliance.

Instead, ROSE-T should be regarded as a decision-support and experimentation tool. By reproducing the verification workflow in a controlled emulated environment, it allows network operators to assess whether their Autonomous System is likely to satisfy the technical requirements evaluated during a MANRS assessment, while identifying potential weaknesses before deploying changes to the production network.

This capability is particularly valuable because several MANRS verification

procedures are inherently intrusive. Evaluating routing policies, source address validation mechanisms, or filtering behaviour often requires the injection of test traffic and the observation of the network response under conditions that may not naturally occur during normal operation. Executing these tests directly on a production infrastructure without prior validation may expose configuration errors, trigger unexpected routing behaviour, or, in the worst case, temporarily affect the availability of network services.

By leveraging network emulation, ROSE-T enables operators to safely reproduce these verification procedures in an isolated environment, evaluate alternative routing policies and experiment with different mitigation strategies before applying them to the operational infrastructure. Although the framework cannot guarantee that a candidate Autonomous System will successfully complete the official MANRS validation process, it significantly reduces the uncertainty associated with that process and provides operators with an opportunity to detect and resolve potential issues before they impact a live network.

The objective of this chapter is to describe the original design of ROSE-T before the extensions introduced in this thesis. The following sections present the architecture of the framework, its verification pipeline and the main assumptions and limitations that motivated the work described in the remainder of this dissertation.

4.1 Original architecture

ROSE-T is entirely developed in Python and follows a modular software architecture, described in Figure 4.2, in which each component is responsible for a well-defined stage of the verification process. Rather than implementing a monolithic application, the framework is organized as a collection of independent modules that cooperate to transform the configuration of a candidate Autonomous System into an executable network scenario and, ultimately, into a MANRS compliance

assessment.



Figure 4.2: Original ROSE-T architecture.

The implementation adopts several well-established software design patterns to improve modularity and extensibility. Support for multiple router vendors is achieved through the *Adapter* pattern, which abstracts vendor-specific configuration formats behind a common interface, allowing the remainder of the framework to operate on a vendor-independent representation of the network regardless of

the syntax of the original configuration files. The *Factory* pattern is employed for the instantiation of protocol-specific objects, while the *Singleton* pattern is used to manage shared resources such as external routing information registries and the network scenario.

At the core of the framework lies the observation that the configuration of a border router contains a significant portion of the information required to reconstruct the surrounding network. Starting from the candidate router configuration, ROSE-T extracts routing sessions, interface addresses, advertised prefixes, routing policies and other protocol-specific parameters. However, router configurations alone do not provide a complete description of the operational environment: they neither describe the routing behaviour of neighbouring Autonomous Systems nor the routing information visible from the global Internet.

To compensate for this lack of information, the framework integrates two additional external data sources. MRT Routing Information Base (RIB) dumps provide a snapshot of the routes observed on the public Internet and are used to infer the external topology surrounding the candidate AS. Internet Routing Registry (IRR) records complement the local configuration by providing the routing policies publicly declared by network operators, enabling consistency checks between the advertised operational behaviour and the information registered in public databases.

The information extracted from both local configurations and external sources is consolidated into a common internal topology model representing routers, interfaces, BGP sessions, prefixes and Autonomous System relationships. This topology serves as the central data structure shared by all subsequent components of the framework.

It is worth emphasizing that the objective of the generated network scenario is not to create a complete digital twin of the original infrastructure. Reproducing every router, link, protocol and operational detail of a production network would require information that is generally unavailable and would unnecessarily increase

the complexity of the emulation. Instead, ROSE-T reconstructs only those aspects of the network that are relevant to the execution of the MANRS verification procedures. This design choice represents a deliberate trade-off between realism and practicality: the emulated scenario is intentionally limited to the information required to reproduce the routing behaviour that influences the outcome of the verification actions, while components that do not affect the evaluated MANRS requirements are omitted or simplified.

The most important aspect preserved by the framework is the behaviour of the candidate Autonomous System itself. The router under verification executes the original vendor configuration supplied by the network operator without modification, ensuring that routing policies, filtering rules, prefix advertisements and forwarding decisions correspond exactly to those that would be observed in the production environment; to achieve this ROSE-T utilises another open-source tool described in Section 4.1.1.

The surrounding Internet, on the other hand, is reconstructed using simplified models. Neighbouring Autonomous Systems are represented by a single router whose configuration is synthesized automatically from the routing information extracted during the topology reconstruction phase. Since the configurations of external networks are not available, these routers are generated using FRRouting (FRR), a vendor-agnostic routing platform capable of reproducing the BGP behaviour required for the verification process.

The emulated scenario also includes an additional synthetic Autonomous System representing the rest of the Internet. Rather than reproducing the global Internet topology, this AS acts as an upstream transit provider advertising a default route toward all external Autonomous Systems, considerably simplifying the emulated environment while preserving end-to-end connectivity and enabling the candidate AS to interact with a realistic approximation of the Internet during the verification process.

By combining the original configuration of the candidate router with an ab-

stract but behaviourally consistent representation of the surrounding network, ROSE-T provides an execution environment that is sufficiently realistic for MANRS validation while remaining lightweight, reproducible and fully automated.

4.1.1 VRNetLab

Running the original vendor configuration on the candidate router introduces a fundamental constraint. Network Operating Systems such as Cisco IOS-XR, Juniper Junos and MikroTik RouterOS are not designed to run natively on a standard Linux kernel. Unlike FRRouting, which operates as a user process on top of Linux and can therefore be packaged directly into a standard Docker container, proprietary vendor NOSes¹ require their own dedicated operating system environment and cannot be executed within a conventional container image.

To overcome this limitation, ROSE-T integrates VRNetLab [L⁺16], an open-source framework that enables vendor virtual machine images to be packaged and executed inside Docker containers. Rather than running the NOS directly as a container process, `vrnetlab` launches a QEMU/KVM virtual machine instance within the container. The vendor-provided disk image is booted inside this virtual machine, ensuring that the NOS operates in its native environment and that all vendor-specific behaviours, including routing protocol implementations, configuration parsing and packet forwarding logic, are preserved exactly as they would be in a production deployment.

From the perspective of the container runtime and Kathará, a `vrnetlab`-based container behaves identically to any other container: it exposes standard network interfaces and participates in the lab topology without requiring special treatment. This transparency is achieved by bridging the container's Ethernet

¹NOS (Network Operating System): The specialised operating system that runs on a router or switch. Unlike a general-purpose OS such as Linux, a NOS is designed specifically for networking functions: interface management, routing protocols, packet forwarding and configuration via a proprietary CLI. Examples include IOS-XR (Cisco), Junos (Juniper) and RouterOS (MikroTik), each with its own internal architecture, configuration format and vendor-specific behaviour.

interfaces directly to the QEMU tap interfaces of the virtual machine using Linux traffic control mechanisms, so that packets arriving on the container's eth interfaces are transparently forwarded to the corresponding virtual NICs of the NOS. As a result, Kathará can connect vnetlab-based containers to collision domains and other devices using the same mechanisms it employs for lightweight Linux containers, without any modification to the lab topology definition.

This design is particularly significant for the integrity of the verification process. By executing the candidate router on the actual vendor NOS rather than on a software approximation, ROSE-T ensures that the routing policies, filtering rules and forwarding decisions being evaluated are those of the real device. The use of vnetlab therefore eliminates an entire class of potential discrepancies between the emulated behaviour and the production behaviour, which would otherwise undermine the validity of the compliance assessment.

4.2 Verification pipeline

The modular architecture presented in the previous section defines the responsibilities of the individual components of ROSE-T. During the execution of the framework, these components cooperate according to a well-defined verification pipeline that transforms a router configuration into an executable emulated network and, finally, into a MANRS compliance assessment. The pipeline begins with the configuration of the candidate Autonomous System together with a limited set of external information sources. These inputs are progressively processed to reconstruct the surrounding network, instantiate an emulated laboratory, execute the verification procedures and collect the results. Figure 4.3 summarizes the main stages of this workflow.

The first step consists of loading the candidate description, which specifies the router configuration to be analysed as well as the external resources required during the execution. Besides the router configuration itself, the framework loads

an MRT Routing Information Base (RIB) dump containing a snapshot of the global routing table. Routing policies for neighbouring Autonomous Systems are retrieved directly from public Internet Routing Registry (IRR) databases during the topology reconstruction phase.

Once all the required inputs have been collected, the candidate configuration is parsed and translated into a vendor-independent internal representation. At this stage, ROSE-T extracts all the information that can be directly inferred from the configuration, including interface addresses, BGP sessions, advertised prefixes, routing policies and protocol parameters. Since this information alone is insufficient to reconstruct the operational environment, it is combined with the external routing data obtained during the previous step.

Using this unified representation, the framework reconstructs the network topology surrounding the candidate Autonomous System. Neighbouring Autonomous Systems are identified, their relationships are inferred and the minimum amount of information required to reproduce the routing behaviour relevant to the verification process is generated. As discussed in the previous section, only the candidate router executes its original vendor configuration, while the surrounding network is synthesized automatically using simplified FRRouting-based routers.

The reconstructed topology is then converted into a Kathará laboratory. The corresponding network scenario, including routers, collision domains, startup scripts and routing configurations, is generated automatically and deployed through the Kathará framework. Once all virtual devices have been instantiated, the framework monitors the state of the BGP sessions and waits until all routing protocol instances have converged, ensuring that the network has reached a stable operational state before any verification is attempted.

After convergence, the verification engine executes the MANRS validation actions. Each action performs an independent assessment of a specific operational requirement by interacting with the emulated network, injecting traffic or routing

information when necessary and observing the resulting behaviour. The execution of every action produces a structured outcome, which may indicate successful verification, the presence of a potential issue, or the impossibility of evaluating a specific requirement under the current network conditions.

Finally, once all verification procedures have completed, the framework aggregates the results into a comprehensive report summarizing the outcome of the analysis. The emulated laboratory is subsequently destroyed, restoring the execution environment to its initial state and allowing additional experiments to be performed without affecting previous executions.

The verification actions executed during the final stage of the pipeline constitute the core of the validation process and are described in detail in the following sections.

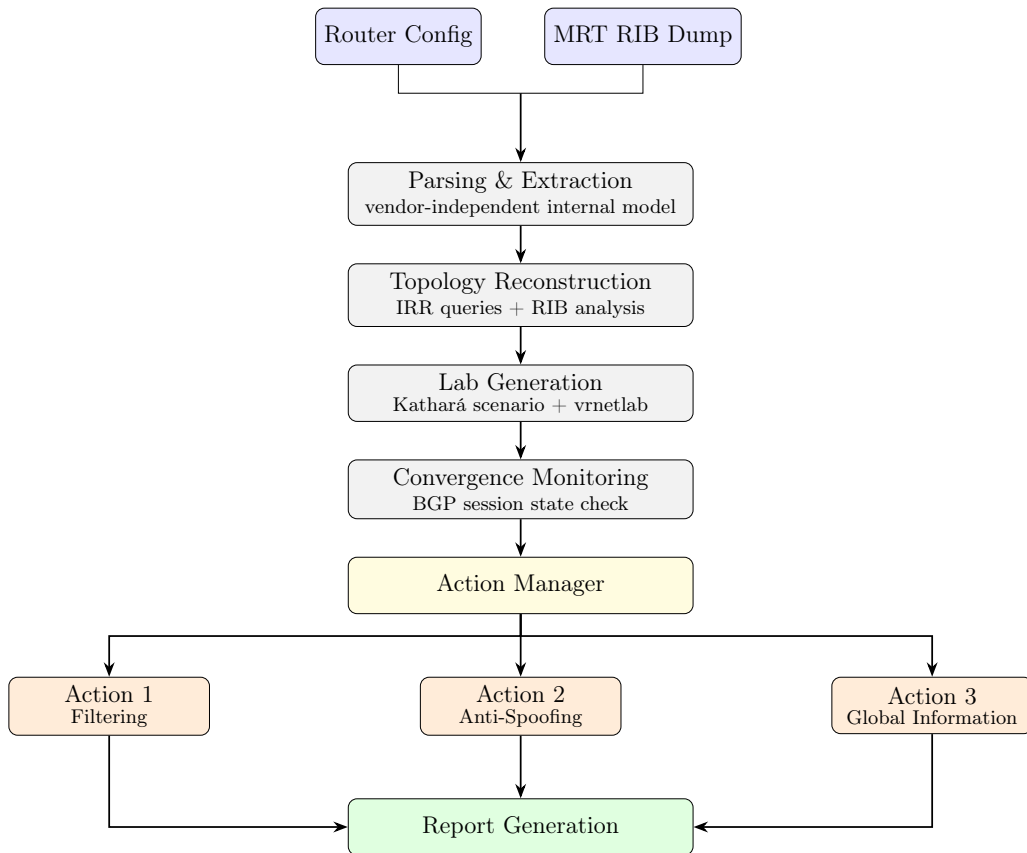


Figure 4.3: ROSE-T verification pipeline.

4.3 Verification Actions

As discussed in Chapter 2.3, the MANRS initiative defines four operational Actions that collectively aim to improve the security and reliability of inter-domain routing. ROSE-T is designed to automatically verify those requirements that can be assessed through network emulation and behavioural testing.

To achieve this objective, the framework implements a modular verification engine based on independent components called *Actions*. Each Action is responsible for validating a specific security property and operates on the reconstructed emulated network without depending on the execution of the others. This modular organization simplifies the extension of the framework while allowing every verification to focus on a single aspect of routing security.

The current implementation of ROSE-T supports three verification Actions, corresponding to three of the four MANRS Actions defined for network operators. The remaining MANRS Action, *Coordination*, is intentionally not included. Unlike the other requirements, Coordination concerns organizational practices, such as maintaining up-to-date contact information and responding to operational incidents, rather than measurable network behaviour. Consequently, it cannot be meaningfully evaluated through automated experimentation on an emulated network.

It is also important to emphasize that ROSE-T does not attempt to provide a complete implementation of the corresponding MANRS Actions. Instead, each verification focuses on the technical aspects that can be objectively assessed through network emulation and external routing information. Administrative procedures, operational processes and documentation requirements remain outside the scope of the framework.

Table 4.1 summarizes the relationship between the verification modules implemented by ROSE-T and the corresponding MANRS Actions.

The following sections describe each verification Action, explaining its objec-

ROSE-T Action	MANRS Action	Verified Aspect	Not Verified
Route Leak	Action 1 - Filtering	Route leaks	IRR/RPKI filtering
Anti-Spoofing	Action 2 - Anti-Spoofing	Forwarding spoofed packets	×
Global Information	Action 3 - Global Validation	Transit providers	All IRR objects
×	Action 4 - Coordination	Not applicable	Operational contacts

Table 4.1: Relationship between the verification Actions implemented by ROSE-T and the MANRS Actions.

tive, execution strategy and evaluation criteria.

4.3.1 Route Leak

The Route Leak action implements the automated verification of the first MANRS Action, *Filtering*. Its objective is to determine whether the candidate Autonomous System correctly applies export policies that prevent customer-learned routes from being propagated to providers or peers, thus avoiding the generation of route leaks.

Rather than inspecting the router configuration to determine whether suitable routing policies are present, ROSE-T evaluates the operational behaviour of the emulated network. This design choice makes the verification independent of the configuration language adopted by the network operator and focuses exclusively on the observable effects of the routing policies after BGP convergence.

The verification procedure begins by identifying the external BGP sessions established by the candidate Autonomous System and classifying them according to their business relationship. At least one provider and one customer are required to perform the experiment, since route leaks are defined by the incorrect prop-

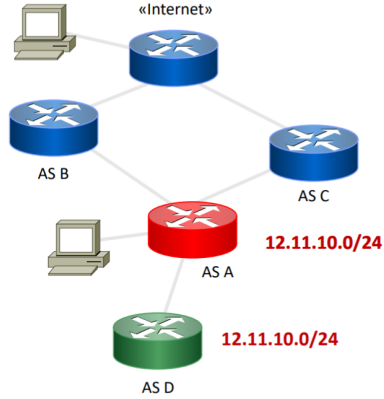


Figure 4.4: Example scenario compliant with Filtering Action.

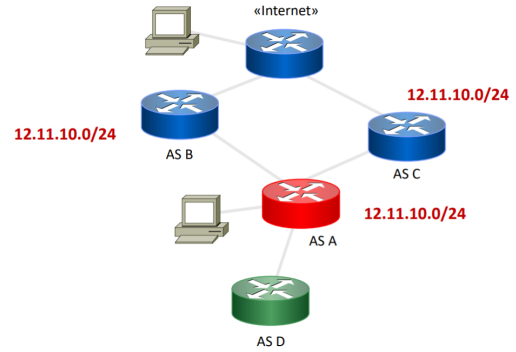


Figure 4.5: Example scenario not compliant with Filtering Action.

agation of routes between different relationship categories. If these conditions are not satisfied, the verification cannot be executed and a warning is reported instead of a validation result.

Once the topology has converged, the customer router originates a dedicated test prefix that is not initially present in the network. This announcement is propagated to the candidate Autonomous System through the customer-provider BGP session. According to the valley-free routing model discussed in Section 2.1.1, the candidate router should learn the route and make it available to its own customers, but it must not advertise it to providers or peers.

To verify this behaviour, ROSE-T does not inspect the candidate router's routing policies directly. Instead, it interacts with the neighbouring routers after BGP convergence has been reached. The framework queries the routing tables of the provider and peer routers to determine whether the injected prefix has been installed as a valid BGP route. If the route is absent from these routing tables, the candidate AS has correctly filtered the announcement and the verification succeeds, an example of this behaviour is presented in Figure 4.4. Conversely, if the test prefix appears in the routing table of a provider or a peer, this demonstrates that the candidate Autonomous System has exported a customer-learned

route where it should not have, constituting a route leak, as shown in Figure 4.5. In this case, the action reports an error.

This behavioural verification offers two important advantages. First, it directly validates the effect of the routing policies instead of attempting to interpret vendor-specific configuration languages. Second, it naturally accounts for the interaction between multiple routing policies, route maps, prefix lists and BGP attributes, since the final routing state reflects the combined effect of all these mechanisms after protocol convergence.

An important characteristic of this approach is that different routing policy implementations leading to the same operational behaviour are considered equally valid. As a consequence, ROSE-T can verify heterogeneous vendor configurations without requiring any knowledge of the syntax or implementation details of the underlying network operating system.

The current implementation focuses specifically on the behavioural aspect of MANRS Filtering related to route leak prevention. Other requirements encompassed by the MANRS Filtering Action, such as prefix filtering based on IRR or RPKI information, validation of *bogon prefixes*², or additional routing hygiene mechanisms, are outside the scope of this verification and are therefore not evaluated by the framework.

4.3.2 Anti-Spoofing

The Anti-Spoofing action implements the automated verification of the second MANRS Action, *Anti-Spoofing*. Its objective is to determine whether the candidate Autonomous System correctly prevents the forwarding of packets carrying forged source IP addresses while still allowing legitimate traffic to traverse the

²Bogon prefixes are IP address ranges that should never appear in public Internet routing tables. They include unallocated address space, addresses reserved for private use (such as 10.0.0.0/8 and 192.168.0.0/16 as defined in [MKR⁺96], loopback addresses, and other special-purpose ranges defined by the IETF. Advertising or accepting bogon prefixes in BGP sessions is generally indicative of misconfiguration or malicious activity.

network.

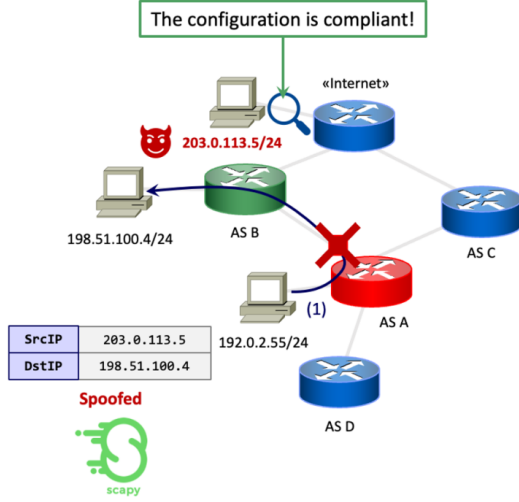


Figure 4.6: Example scenario compliant with Anti-Spoofing Action.

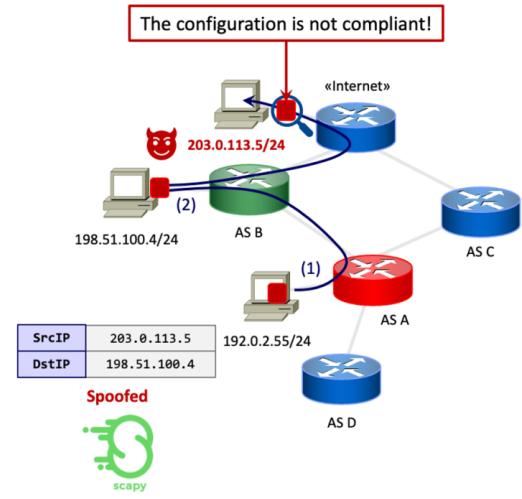


Figure 4.7: Example scenario not compliant with Anti-Spoofing Action.

As for the previous action, ROSE-T performs a behavioural verification instead of inspecting the router configuration. The framework does not attempt to determine whether source address validation is implemented through uRPF, ACLs, or any other mechanism. Instead, it evaluates the observable forwarding behaviour of the network by injecting both legitimate and spoofed traffic into the emulated topology.

To execute the experiment, ROSE-T automatically creates a set of lightweight client hosts within the emulated laboratory. One client is attached to the candidate router using its first available interface and acts as the source of the test traffic. Additional clients are connected to each provider router, while another client is attached to the Internet node automatically generated during the topology reconstruction process. These hosts do not participate in routing; their sole purpose is to generate and observe traffic during the verification procedure.

Each observation client executes a lightweight Python script that continuously captures incoming packets. These scripts allow the framework to deter-

mine whether the transmitted traffic reaches the expected destination without requiring any manual interaction with the laboratory.

Once the network has reached BGP convergence, the client connected to the candidate router transmits two different traffic flows. The first consists of legitimate packets whose source address belongs to the subnet assigned to the client itself. The second consists of packets carrying a forged source address that does not belong to the originating network, thereby simulating an IP spoofing attack. Both flows are generated and sent using *Scapy*, which allows ROSE-T to craft packets with arbitrary header fields and inject them directly into the emulated network; Scapy is presented in [Bio07].

The outcome of the verification is determined by combining the observations collected by the monitoring clients. A successful verification requires that the legitimate traffic reaches the observation points while the spoofed traffic is discarded before leaving the candidate Autonomous System, as in Figure 4.6. If both traffic flows are blocked, the test also fails, since the candidate network would be preventing valid communication in addition to malicious traffic. Similarly, if the spoofed packets reach one of the observation clients, the framework reports an error because the candidate AS is forwarding traffic with invalid source addresses; a non-compliant scenario is shown in Figure 4.7.

This dual validation strategy is particularly important because it evaluates both security and connectivity. The objective is not simply to block traffic, but to ensure that source validation mechanisms correctly distinguish between legitimate and forged packets without disrupting normal network operation.

As with the Route Leak Prevention action, the verification remains completely independent of the underlying implementation. Different anti-spoofing mechanisms may produce the same forwarding behaviour and ROSE-T considers all of them equally valid provided that legitimate packets are successfully forwarded and spoofed packets are consistently discarded.

The current implementation focuses exclusively on the forwarding behaviour

required by the MANRS Anti-Spoofing Action. It therefore verifies the effectiveness of the deployed protection mechanism without attempting to identify which specific technique is responsible for the observed behaviour.

4.3.3 Global Information

The Global Information action implements the automated verification of the third MANRS Action, *Global Validation*. Its objective is to determine whether the information published by the candidate Autonomous System in Internet Routing Registries (IRRs) is consistent with the routing information that can be observed on the Internet.

Unlike the previous actions, which actively interact with the emulated network by injecting routes or packets, this verification is entirely data-driven. The framework compares routing information obtained from two independent sources: the routing policies published in the IRR database and the routes observed through MRT RIB dumps collected from public route collectors.

The verification begins by retrieving the import policies associated with the candidate Autonomous System from the IRR database. Rather than analysing the complete policy specification, ROSE-T focuses exclusively on import rules of the form:

```
import: from ASX accept ANY
```

Such rules indicate that the candidate AS accepts arbitrary routes from the neighbouring Autonomous System and therefore treats it as an upstream transit provider. The framework extracts all Autonomous Systems appearing in these rules and builds a set representing the transit providers declared in the IRR.

The second phase analyses the MRT RIB dump. ROSE-T retrieves all routes originated by the candidate AS and inspects their AS-PATH attributes. For each route, the framework identifies the Autonomous System immediately preceding

the candidate AS in the AS-PATH. This AS is interpreted as the transit provider through which the route has been propagated towards the route collector.

For example, consider the following AS-PATH:

```
64496 3356 65001
```

where AS65001 is the candidate AS. In this case, AS3356 is identified as the transit provider because it is the Autonomous System directly adjacent to the candidate within the observed path.

To reduce ambiguities, ROSE-T ignores routes received from collectors belonging to well-known Tier-1 networks and discards paths containing fewer than three Autonomous Systems. The resulting set represents the transit providers observed on the Internet.

Once both sets have been constructed, the framework performs a simple set comparison. A successful verification requires that every transit provider declared in the IRR also appears in the routing information and that every transit provider observed in the routing information is declared in the IRR.

Two types of inconsistencies may be detected. The first occurs when a transit provider is observed in the routing information but is missing from the IRR records. This indicates that the operational connectivity of the Autonomous System has changed without updating the corresponding registry information. The second occurs when a transit provider is declared in the IRR but never appears in the routing information, suggesting that the published routing policy no longer reflects the actual state of the network.

For instance, suppose that the candidate AS declares the following IRR policy:

```
import: from AS3356 accept ANY
import: from AS1299 accept ANY
```

while the MRT RIB dump reveals routes containing the paths

64496 3356 65001

64497 2914 65001

In this case, AS2914 is observed on the Internet but is not declared in the IRR, while AS1299 is declared but never observed. Both inconsistencies would be reported by the framework as validation errors.

It is important to note that this action does not attempt to validate the correctness of the complete routing policy published by the operator. Instead, it verifies the consistency between the set of transit providers declared through the IRR and the set inferred from publicly observable routing information. This abstraction significantly simplifies the verification process while still addressing the core objective of the MANRS Global Validation action.

The current implementation focuses exclusively on transit-provider consistency and does not verify other aspects of routing registries, such as route objects, AS-SET definitions, RPKI Route Origin Authorizations (ROAs), or the completeness of the information published by the operator.

4.4 Limitations

The original version of ROSE-T successfully demonstrated the feasibility of automating MANRS compliance verification through network emulation. However, several design decisions adopted to simplify the verification process inevitably restrict the applicability of the framework in realistic operational environments.

Some limitations are specific to individual verification actions and have already been discussed in the corresponding sections. In particular, the implemented actions do not cover the entire scope of the MANRS recommendations and focus only on those aspects that can be evaluated automatically through emulation. Beyond these action-specific constraints, the original framework exhibits three major architectural limitations: its dependence on external routing

information sources, its restricted support for router vendors and its assumption that a single router can adequately represent an entire Autonomous System.

4.4.1 Dependence on External Routing Information

Unlike the Route Leak and Anti-Spoofing actions, which can be fully evaluated within an emulated environment, the Global Information action depends on external data sources. The verification requires both routing registry information retrieved from IRR databases and MRT RIB dumps collected from public route collectors.

As a consequence, the framework cannot operate exclusively on synthetic laboratory scenarios. Even when the candidate network is entirely emulated, the Global Information verification still relies on information gathered from the real Internet. This dependency prevents the creation of completely self-contained experimental environments and limits the possibility of performing reproducible offline evaluations.

Furthermore, the correctness of the verification depends on the availability and quality of the external datasets. Missing routing registry entries, outdated MRT dumps, or inconsistencies in publicly available information may legitimately lead to the failure of the Global Information action and, at the same time, affect the reconstruction of the emulated network, since part of the surrounding topology is generated from these data sources. Such behaviour does not represent a limitation of the framework, but rather reflects the actual conditions that would also influence an official MANRS assessment.

While this design is appropriate for validating operational Autonomous Systems connected to the Internet, it reduces the flexibility of the framework when used as a pure experimentation platform.

4.4.2 Limited Vendor Support

The original implementation of ROSE-T supports a limited set of router operating systems. Candidate configurations can only be imported from Cisco IOS, JunOS and MikroTik devices through dedicated configuration adapters.

Although FRRouting (FRR) is extensively used within the emulated scenarios generated by the framework, it is employed exclusively to recreate neighbouring Autonomous Systems and auxiliary devices. The framework is not capable of directly evaluating MANRS compliance starting from an FRRouting configuration.

Consequently, networks built around unsupported routing platforms cannot be analysed without implementing additional vendor-specific adapters. While the adapter-based architecture facilitates future extensions, the practical applicability of the framework remains constrained by the availability of configuration parsers for the target platform.

4.4.3 Single Router Assumption

The most significant limitation of the original ROSE-T architecture is the assumption that a single router can represent the behaviour of an entire Autonomous System.

The verification process starts from the configuration of one candidate router, which is assumed to be sufficient to reconstruct the relevant characteristics of the surrounding network. All MANRS checks are subsequently performed against this router, while the rest of the Autonomous System is not explicitly modelled.

This assumption simplifies topology reconstruction and significantly reduces the complexity of the emulated environment. However, it does not accurately reflect the architecture of modern Autonomous Systems, which commonly consist of multiple interconnected routers, often distributed across different locations and performing distinct operational roles.

Most importantly, compliance of a single router does not imply compliance of

the entire Autonomous System. Security mechanisms such as route filtering, anti-spoofing policies, or routing policy enforcement may be deployed inconsistently across different routers. As a result, a verification performed on one router may produce a positive outcome even though another router belonging to the same Autonomous System would fail the same validation.

The limitation becomes particularly evident for the Anti-Spoofing action, where traffic filtering may occur at different ingress points of the network and for Route Leak prevention, where export policies can differ across border routers. In such scenarios, evaluating a single device provides only a partial view of the overall security posture of the Autonomous System.

This limitation motivated the primary contribution of the present work, namely the extension of ROSE-T from a single-router verification model to a framework capable of representing and validating multi-router Autonomous Systems.

Chapter 5

Extending ROSE-T

The limitations discussed in the previous chapter motivated a substantial redesign of several components of the original ROSE-T architecture.

This work introduces four main contributions. First, support for FRRouting configurations was added, allowing the framework to directly evaluate networks based on one of the most widely adopted open-source routing platforms. Second, a mechanism for providing local Global Information data was introduced, enabling fully self-contained laboratory experiments without relying on external Internet resources. Third, the topology reconstruction process and the verification actions were redesigned to support Autonomous Systems composed of multiple routers rather than a single candidate device. Finally, these extensions were complemented by the development of ROSET-Adapter, a tool that automates the extraction of network information and the generation of the inputs required by ROSE-T.

All the extensions presented in this chapter have been implemented as part of a public fork of the original ROSE-T project. In order to preserve compatibility with the upstream codebase while enabling independent development, the modified version of ROSE-T has been released as an open-source fork together with the auxiliary tool ROSET-Adapter. Both repositories are publicly available and contain the complete implementation used throughout the experiments discussed

in this thesis: [Sac26a], [Sac26b]. The source code can therefore be inspected, reproduced, and further extended by other researchers and network operators interested in MANRS compliance verification. The fork-based development model also facilitates the integration of future improvements into the original project whenever appropriate.

The following sections describe each contribution in detail, discussing both the implementation choices and their impact on the overall verification workflow.

5.1 FRR support

5.1.1 Motivation

As discussed in the previous chapter, the original version of ROSE-T only supported a limited set of router operating systems for candidate configurations, namely Cisco IOS, JunOS, and MikroTik. While these platforms are widely adopted in production environments, the absence of support for FRRouting (FRR) significantly limited the applicability of the framework in experimental and academic settings.

FRR is one of the most widely adopted open-source routing suites and provides implementations of several routing protocols, including BGP, OSPF, IS-IS, and RIP. Due to its flexibility, active development, and ease of deployment, FRR has become a de facto standard for network experimentation and research environments.

Its relevance is particularly evident in the context of Kathará. Most routing laboratories distributed with Kathará rely on FRR-based routers, and the framework itself provides native support for FRR containers. Consequently, although the original version of ROSE-T used FRR extensively to emulate neighbouring Autonomous Systems, it was not possible to directly evaluate the compliance of an Autonomous System whose actual configuration was implemented using FRR.

This limitation reduced the usefulness of the framework for network operators, researchers, and students wishing to evaluate MANRS compliance within fully open-source environments. Supporting FRR therefore represented a natural extension of ROSE-T, enabling the verification of networks built entirely with technologies that are freely available and easily reproducible.

5.1.2 Configuration Parsing

The introduction of FRR support required the implementation of two new components: an FRR-specific configuration parser and a corresponding `FrrConfiguration` class.

The architecture of ROSE-T already employed an Adapter pattern to isolate vendor-specific configuration formats from the rest of the verification framework. Each supported vendor is represented by a dedicated parser responsible for translating the native configuration syntax into a common internal representation used by the topology reconstruction and validation components.

Thanks to this design, adding support for a new routing platform did not require modifications to the verification pipeline itself. Instead, it was sufficient to implement the vendor-specific components while preserving the interfaces defined by the abstract base classes.

More specifically, the new `FrrParser` class extends the generic parser hierarchy and is responsible for extracting all information required by ROSE-T from FRR configuration files, including BGP sessions, routing policies, prefix filters, route maps, and Autonomous System identifiers. The resulting data are stored in an instance of the `FrrConfiguration` class, which specializes the generic `VendorConfiguration` abstraction already used throughout the framework.

Once translated into the internal representation, FRR configurations become indistinguishable from configurations originating from other vendors. As a consequence, all subsequent phases of the verification process, including topology

reconstruction, scenario generation, and MANRS validation, can operate without any vendor-specific logic.

The implementation therefore confirms the effectiveness of the architectural choices adopted in the original ROSE-T design. The use of abstraction layers together with the Adapter pattern significantly reduces the effort required to support additional vendors. Although only FRR support was added in the context of this work, the same approach could be applied to integrate further routing platforms with limited impact on the rest of the codebase.

5.1.3 Startup File Specification

Supporting FRR required more than simply introducing a new configuration parser. During the implementation, it became evident that certain aspects of a router's behaviour cannot be expressed directly through the FRR configuration language and must instead be configured at the operating system level.

Unlike proprietary routing platforms, where networking features are often integrated into the router configuration itself, FRR focuses primarily on routing protocols and routing policies. As a consequence, several functionalities that may influence MANRS compliance verification are configured externally through operating system commands executed during system startup.

Examples include the configuration of firewall rules, kernel networking parameters, packet filtering mechanisms, and Reverse Path Forwarding (RPF). These settings are not represented within the FRR configuration files and therefore cannot be reconstructed by parsing them alone.

This limitation is particularly relevant in the context of ROSE-T. Consider, for example, the Anti-Spoofing verification. A network operator may enforce source address validation through Linux kernel mechanisms such as Reverse Path Forwarding. Since these settings are not part of the FRR configuration, a parser operating exclusively on routing configuration files would be unable to determine whether the protection is actually enabled.

To address this issue, the internal representation of candidate routers was extended with support for startup scripts. Each candidate router can now optionally specify an associated startup file containing arbitrary shell commands that are executed when the emulated router is instantiated within the Kathará laboratory.

During the deployment phase, ROSE-T automatically applies both the parsed FRR configuration and the corresponding startup script. This approach makes it possible to reproduce not only the routing behaviour of the candidate router but also the operating system configurations that may affect the outcome of the MANRS validation process.

Beyond enabling support for FRR-based networks, this extension improves the overall expressiveness of the framework. The startup file mechanism provides a generic way to represent configuration aspects that fall outside the scope of routing protocols while remaining relevant to security validation. As a result, future extensions can leverage the same mechanism to model additional operating system features without requiring modifications to the routing configuration parsers themselves.

5.2 Global Information File

5.2.1 Motivation

As discussed in the previous chapter, one of the main limitations of the original ROSE-T implementation was the dependence of the Global Information action on external Internet resources. While the Route Leak and Anti-Spoofing actions can be executed entirely within an emulated environment, the Global Information verification requires both routing registry information and MRT RIB dumps collected from public route collectors.

This design is appropriate when evaluating operational Autonomous Systems connected to the Internet, which is the primary use case envisioned by the origi-

nal authors. However, it introduces significant limitations when the framework is used for experimentation and research purposes. In particular, it becomes impossible to create fully synthetic laboratory scenarios whose expected behaviour is completely under the control of the experimenter. Even if the network topology is entirely emulated, the verification process still depends on information retrieved from external IRR databases and publicly available routing data.

This limitation became particularly evident during the development and validation of the extensions presented in this thesis. In order to evaluate the behaviour of the framework under controlled conditions, it was necessary to construct laboratory scenarios specifically designed to trigger or avoid individual MANRS violations. Such scenarios do not correspond to real Autonomous Systems and therefore cannot be associated with meaningful IRR entries or publicly observable routing information.

Furthermore, the generation of suitable MRT RIB dumps proved to be a non-trivial problem. During the initial stages of development, custom dumps had to be manually constructed in order to reproduce specific routing situations and test the correctness of the Global Information verification logic. As will be discussed later in the chapter dedicated to ROSET-Adapter 5.4, this process was subsequently automated. Nevertheless, the need to manually create these datasets highlighted the necessity of reducing the dependence on external routing information sources.

To address this limitation, the Global Information subsystem was extended to support local information repositories. Rather than retrieving routing relationships exclusively from public IRR databases, ROSE-T can now load a JSON file describing both the relationships between Autonomous Systems and the transit information required by the Global Information action.

A simplified example of the file format is shown below:

```
{  
  "AS103": {  
    "AS101": "provider",
```



```
"AS102": "customer",
"AS104": "peer"
},
"as_rules": {
  "AS103": {
    "transits": [101]
  }
}
```

The file contains two distinct types of information.

The first section describes the business relationships between Autonomous Systems. These relationships are used throughout the framework whenever provider, customer, or peer information is required. In the previous example, the entry

```
"AS103": {
  "AS101": "provider",
  "AS102": "customer",
  "AS104": "peer"
}
```

indicates that AS101 acts as a provider for AS103, AS102 is a customer and AS104 is a peer. This information can therefore be used in place of data retrieved from external routing registries and needs to be present for each experiment to build a meaningful representation of the original AS.

The second section, `as_rules`, provides the transit information required by the Global Information action. In the example above, AS103 declares AS101 as one of its transit providers. Internally, this information is translated into the same representation used when parsing IRR import rules of the form:

```
import: from AS101 accept ANY
```

As a consequence, the Global Information action can operate without any modification to its verification logic. The action simply receives the transit providers extracted from the local file rather than from the RIPE database. This section is not strictly necessary; if it is not included, it will be treated as though the candidate AS does not publicly declare any transit providers, and the global information test will be carried out on the basis of this.

5.2.2 Implementation Details

From an implementation perspective, the extension was designed to minimize modifications to the existing ROSE-T architecture. Rather than introducing a separate component for handling local information, the existing `RipeDb` class was extended to support both external and local data sources.

The original `RipeDb` component already acted as a centralized access point for IRR information and followed the Singleton pattern, ensuring that routing registry information was retrieved and cached through a single shared instance. The same design was preserved in the extended version. Two additional internal data structures were introduced: one storing locally defined Autonomous System relationships and another storing locally defined transit information.

At startup, ROSE-T can optionally load a JSON file containing the additional information. The file is parsed once and stored in memory, making subsequent lookups independent from external services. This approach also avoids repeated file accesses during verification.

The first extension concerns relationship retrieval. When ROSE-T needs to determine the business relationship between two Autonomous Systems, the framework first checks whether a local relationship has been defined. Relationships are internally translated into the same numerical representation already used by the topology reconstruction process, namely provider, customer, and peer. As a consequence, the rest of the framework can continue operating without modifications.

The second extension concerns the retrieval of IRR routing policies used by

the Global Information action. In the original implementation, ROSE-T queried the RIPE database and extracted `import` and `export` policies from the returned RPSL objects. The Global Information action subsequently analysed these rules to identify transit providers declared by the candidate Autonomous System.

To preserve compatibility with the existing verification logic, locally specified transit providers are automatically converted into synthetic import policies having the same structure as the RPSL entries normally retrieved from the RIPE database. For example, a transit declaration such as

```
"transits": [101]
```

is internally translated into

```
from AS101 accept ANY
```

which is exactly the format expected by the original parser. Consequently, no modifications were required in the Global Information action itself. The action continues to process import rules exactly as before, regardless of whether they originate from the RIPE database or from a local file.

Another important design choice concerns backward compatibility. Whenever no local information is available for a given Autonomous System, the component automatically falls back to the original behaviour and queries the RIPE database. Existing ROSE-T deployments therefore continue to operate unchanged, while experimental scenarios can selectively override only the information that needs to be controlled.

This solution significantly reduced the implementation effort while maintaining a clean separation between data acquisition and verification logic. More importantly, it allowed all existing actions to benefit from locally defined information without requiring modifications to their internal algorithms.

5.3 Multi-router AS support

As discussed in the previous chapter, one of the most significant limitations of the original ROSE-T architecture was the assumption that an Autonomous System could be represented by a single router configuration. While this simplification greatly reduced the complexity of topology reconstruction and scenario generation, it does not accurately reflect the structure of modern operational networks.

In practice, Autonomous Systems are commonly composed of multiple routers fulfilling different roles. Border routers establish external BGP sessions with providers, customers, and peers, while internal routers exchange routing information through iBGP and Interior Gateway Protocols. Security mechanisms and routing policies may therefore be distributed across several devices rather than concentrated on a single router.

Under these conditions, validating only one router does not provide a complete view of the security posture of the Autonomous System. A border router may correctly implement route filtering policies while another router belonging to the same AS does not. Similarly, anti-spoofing protections may be enabled on some ingress points and absent on others. Consequently, the compliance of a single router cannot be considered equivalent to the compliance of the entire Autonomous System.

Supporting multi-router environments therefore represented a fundamental requirement for extending the applicability of ROSE-T to more realistic network scenarios. However, introducing this capability required substantial modifications to several components of the framework. The original topology reconstruction process assumed a single candidate device, the validation actions interacted with a single router, and the emulated laboratory topology was generated around a single configuration file.

The extension presented in this thesis removes this limitation by allowing multiple routers belonging to the candidate Autonomous System to participate

in the reconstruction and verification process. Rather than validating an isolated router, ROSE-T is now able to reconstruct an internal AS topology, deploy multiple candidate devices within the emulated environment, and execute the validation actions while taking their interactions into account.

The following sections describe the main architectural modifications required to achieve this objective.

5.3.1 Topology Builder

Supporting multi-router Autonomous Systems required a substantial redesign of the topology reconstruction process. In the original implementation, the **Topology** component assumed the presence of a single candidate router and generated the emulated environment around that device. As a consequence, the entire Autonomous System was represented by one node connected to neighbouring providers, customers, and peers.

To overcome this limitation, a new candidate model based on the **AsCandidate** abstraction was introduced. Rather than providing a single router configuration, the framework can now receive a collection of routers belonging to the same Autonomous System. Each router is associated with its own vendor-specific configuration and startup script, preserving the routing policies deployed on the devices that actually participate in BGP.

The topology builder was extended accordingly. During initialization, the framework automatically detects whether the execution concerns a traditional single-router scenario or a multi-router candidate. In the latter case, a dedicated reconstruction pipeline is executed, allowing the BGP-relevant portion of the Autonomous System to be explicitly represented within the emulated environment.

A key design choice of this extension is that only routers participating in BGP are virtualized. Internal devices that do not run BGP are intentionally ignored, since the goal of ROSE-T is not to reproduce the full internal structure of the network, but to validate its compliance with MANRS requirements. In

other words, the framework does not aim to build a complete digital twin of the Autonomous System; instead, it reconstructs only the routing elements that are relevant for policy verification and traffic exchange.

The reconstruction process begins by instantiating all candidate BGP routers and creating the corresponding interfaces and collision domains based on the information extracted from their configurations. Unlike the original implementation, where a single node represented the entire candidate AS, each BGP-speaking router is now modelled as an independent entity within the topology graph. This design allows the framework to preserve the distinction between border routers and internal BGP routers while omitting devices that do not contribute to the control plane under test.

Once all candidate routers have been instantiated, the framework infers the relationships associated with their external BGP sessions. As in the original implementation, relationship information can be obtained either from routing registry data or from the local Global Information File introduced in the previous section. This phase identifies providers, customers, and peers and associates the corresponding business relationship with each external session.

The next step consists in reconstructing the internal connectivity of the Autonomous System. This operation relies on the iBGP sessions configured on candidate routers. When two routers advertise iBGP peering using directly connected addresses, the framework identifies the matching endpoints and recreates the corresponding internal link. The resulting connection is represented through a shared collision domain and preserves the addressing information specified in the original configurations.

A particularly important extension concerns support for multi-hop iBGP sessions. Since loopback-based peering is not supported in the current implementation, routers configured with multi-hop iBGP are instead connected to an internal hub that acts as a shared transit segment. In this way, the routers can still exchange BGP traffic through the emulated topology without requiring direct

point-to-point connectivity. To preserve reachability, the topology builder also adds a static route using the IP address of the neighbour specified in the router configuration, ensuring that the peering endpoint can be reached through the hub. This approximation may, however, interact with strict uRPF configurations by altering the expected reverse path towards the iBGP neighbour, potentially affecting the outcome of the Anti-Spoofing verification in topologies employing multi-hop iBGP sessions.

This approach reflects the scope of the framework. ROSE-T does not attempt to reproduce every internal forwarding device or every physical link of the original network. Instead, it virtualizes only the routers that matter for BGP behaviour and compliance verification, while simplifying the rest of the Autonomous System. Direct links are created where the configuration indicates a directly connected iBGP adjacency, whereas multi-hop sessions are mapped onto a shared hub when a direct link cannot be inferred. This provides enough fidelity to reproduce the routing behaviour relevant to MANRS checks without introducing the complexity of a full network twin.

External Autonomous Systems continue to be represented using a simplified model. Each neighbouring AS is instantiated as a single router regardless of its actual internal structure. This choice is intentional and reflects the objectives of the framework. Since the configurations of external routers are generally unavailable, their behaviour is recreated synthetically using automatically generated FRR configurations.

The reconstruction process also preserves the Internet abstraction already present in the original implementation. A dedicated Autonomous System is created to represent global Internet connectivity and originates default routes toward the candidate network. This component acts as an upstream provider and allows the execution of validation tests that require communication with destinations outside the reconstructed topology.

Finally, the topology builder automatically deploys auxiliary client nodes at-

tached to candidate routers and external providers. These clients are later used by the validation actions to generate traffic, perform packet captures, and verify the behaviour of the network under test. While largely transparent to the user, this mechanism constitutes an essential element of the verification workflow and enables several of the experiments discussed in the following chapters.

Figures 5.1 and 5.2 show how the new topology builder works, compared to the previous version, compared to the previous version.

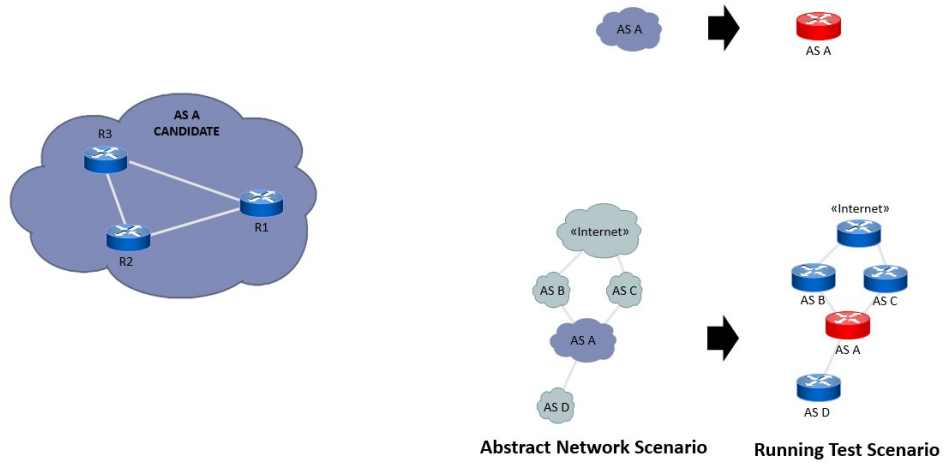


Figure 5.1: Topology reconstruction in old ROSE-T version.

Overall, the new topology builder extends the original ROSE-T architecture from a router-centric model to an Autonomous-System-centric model. Instead of validating an isolated routing device or attempting to emulate the entire internal network, the framework now reconstructs only the BGP-speaking portion of the Autonomous System and the external relationships needed for verification. This keeps the emulated environment lightweight and reproducible while remaining sufficient for MANRS compliance testing.

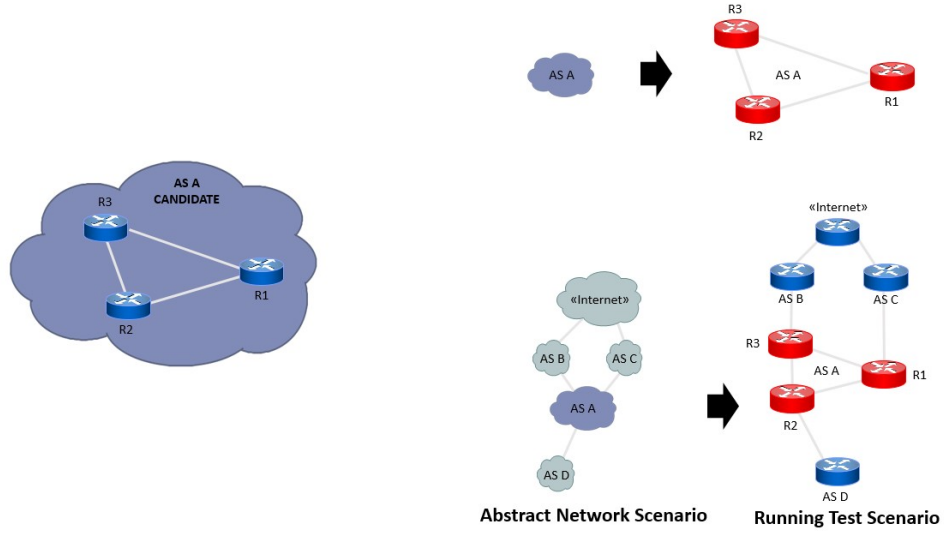


Figure 5.2: Topology reconstruction in new ROSE-T version.

5.3.2 Actions Redesign

Extending the topology builder to support multiple routers within the candidate Autonomous System was not sufficient on its own. The original validation actions had been designed under the assumption that the candidate AS was represented by a single router and therefore interacted with only one device during the verification process.

While some validations naturally operate at the Autonomous System level and therefore remain unaffected by the internal structure of the network, others require the generation and observation of traffic from specific points of the topology. Consequently, several actions had to be redesigned in order to operate correctly in the presence of multiple candidate routers.

The following paragraphs discuss the modifications introduced for each validation action.

Global Information The Global Information action required no substantial modifications. Unlike the other validations, its objective is to verify the consistency between the transit providers declared by an Autonomous System and the transit providers observed from publicly available routing information.

This verification is inherently performed at the AS level rather than at the router level. Regardless of how many routers compose the candidate network, the action ultimately compares information associated with the Autonomous System as a whole. The introduction of multi-router support therefore does not alter either the validation logic or the interpretation of the results.

Similarly, the introduction of the Global Information File described in the previous section did not require any changes to the action itself. The local file was deliberately implemented inside the data acquisition layer so that the action could continue to operate exactly as before. From the perspective of the validation logic, locally provided information and information retrieved from public IRR databases are exposed through the same interface and therefore remain completely transparent to the action implementation.

Anti-Spoofing The Anti-Spoofing action required the most significant redesign among all validation components.

In the original implementation, a single client was attached to the candidate router and used as the source of both legitimate and spoofed traffic. Since only one router existed within the candidate AS, selecting the attachment point was trivial: any interface not involved in an eBGP peering could be used to connect the testing client.

The introduction of multi-router support fundamentally changes this assumption. A compliant Autonomous System must prevent spoofed traffic from leaving the network regardless of the ingress point through which the traffic enters. Testing a single router is therefore insufficient, as another router belonging to the same AS could still allow spoofed packets to escape.

For this reason, the redesigned action creates a dedicated testing client for every candidate router. Each client generates both legitimate and spoofed packets, ensuring that the Anti-Spoofing validation is performed from every possible ingress point represented in the reconstructed topology.

Selecting the attachment point for these testing clients proved to be one of the most challenging aspects of the redesign. In a multi-router topology, simply connecting a host to an arbitrary unused interface is not sufficient. The generated client must be attached to a network that is effectively originated or propagated by the router under test and from which legitimate traffic can reach external destinations.

For each candidate router, the action first attempts to reuse the client interface generated during topology construction. If no suitable interface is available, a dedicated client and a new collision domain are created dynamically. The action then analyses the networks advertised by the candidate AS towards its providers and selects only those that overlap with the subnet associated with the chosen attachment point.

Additional checks are performed to ensure that the selected subnet contains a sufficient number of available addresses. Once a suitable network has been identified, two unused addresses are allocated automatically: one for the testing client and one for the router interface. The client is then configured with a default route pointing towards the candidate router, guaranteeing that legitimate traffic traverses the same forwarding path that would be used by a real host connected to that portion of the network.

This procedure ensures that the Anti-Spoofing validation is executed under realistic routing conditions. Consequently, a failed test can be attributed to the candidate configuration itself rather than to an incorrect placement of the testing infrastructure.

The procedure described above is intentionally conservative. Rather than attaching testing clients to arbitrary interfaces and risking misleading results,

the action only proceeds when all the conditions required to generate meaningful traffic are satisfied. A valid attachment point must belong to a suitable subnet, provide enough available addresses for both the router and the testing client, and allow the generated traffic to be routed correctly through the emulated topology.

As a consequence, certain topologies may not provide a configuration that satisfies all these requirements simultaneously. In such cases, the framework emits a warning and skips the corresponding validation instead of producing a potentially inaccurate result. It is important to note that this situation does not indicate a failure of the Anti-Spoofing verification. Rather, it means that the action was unable to identify a testing environment that guarantees the reliability of the experiment. This design choice prioritizes correctness and reproducibility over execution coverage, avoiding false conclusions derived from artificially constructed or unrealistic testing conditions.

The overall testing methodology remains unchanged. For every candidate router, the action generates both legitimate packets and packets carrying spoofed source addresses. Additional clients attached to providers and to the Internet node execute packet-capture scripts that observe whether the generated traffic reaches the external portions of the topology. A compliant network must simultaneously satisfy two conditions: legitimate traffic must successfully reach the destination, while spoofed traffic must be filtered before leaving the Autonomous System. Both conditions are required in order for the validation to succeed.

Route Leak The Route Leak action required more limited modifications.

The validation logic itself remains unchanged: the objective is still to verify that routes learned from providers are not improperly propagated to other providers or peers in violation of valley-free routing principles. However, the presence of multiple candidate routers introduces additional complexity in identifying where route advertisements should be injected and where they should be observed.

In the original implementation, all routing information passed through a single candidate router, making route propagation paths straightforward to determine. In a multi-router environment, routes may traverse multiple internal devices before reaching an external neighbour, and the validation process must therefore consider the entire candidate AS rather than a single router.

The redesigned implementation leverages the extended topology model introduced by the new topology builder. Since all BGP-speaking routers belonging to the candidate AS are explicitly represented, route advertisements can propagate through the reconstructed iBGP infrastructure exactly as they would within the original network. The action then verifies whether routes originating from providers eventually reach unauthorized external neighbours, independently of which candidate router performs the actual advertisement.

Another important improvement introduced by the multi-router redesign concerns the validation workflow itself. In the original implementation, the absence of route propagation towards unauthorized neighbours was considered sufficient to conclude that no route leak had occurred. However, this assumption may lead to misleading results, as a route can fail to propagate simply because it was never received by the candidate AS in the first place.

To address this issue, the validation process was extended with an additional preliminary verification step. Before checking whether a customer-learned route is incorrectly propagated towards providers or peers, the action first verifies that the route has been successfully received and installed by the candidate AS. Only after confirming the presence of the route within the candidate routing infrastructure does the framework evaluate whether it is subsequently advertised to unauthorized external neighbours.

This modification prevents false positives in which a Route Leak test would appear successful despite underlying connectivity or topology reconstruction issues. The validation therefore becomes a two-stage process: first confirming the correct reception of the customer route and then verifying that the route is not

exported in violation of valley-free routing policies. As a result, the reported outcome more accurately reflects the actual routing behaviour of the candidate Autonomous System.

As a result, the redesigned Route Leak action not only extends the validation process to multi-router Autonomous Systems but also strengthens the reliability of the verification itself. By explicitly confirming that customer-learned routes are successfully received before evaluating their propagation behaviour, the framework can distinguish between genuine policy compliance and failures caused by missing route dissemination or topology reconstruction issues. This additional validation step improves the accuracy of the results while preserving the original security objective of detecting violations of valley-free routing principles.

5.4 ROSET-Adapter

The extensions presented in the previous sections significantly improve the flexibility of ROSE-T, allowing the framework to operate on FRR-based laboratories and multi-router Autonomous Systems. However, during the development and testing phases, another practical challenge quickly emerged.

Although ROSE-T is capable of reconstructing and validating a network scenario, preparing the input required by the framework remains a time-consuming task. Even when working with laboratory environments specifically designed for experimentation, a user must manually collect router configurations, create the candidate description, prepare startup files, generate the Global Information File, and, most importantly, build a routing table dump consistent with the topology under analysis.

This process is particularly cumbersome when the objective is not to validate a production network but rather to experiment with new ideas, evaluate modifications to the framework itself, or generate controlled benchmark scenarios. For this reason, a complementary tool named **ROSET-Adapter** was developed and

released as an open-source project alongside the modified version of ROSE-T.

ROSET-Adapter is specifically designed for laboratory environments built using Kathará and FRR. Instead of manually converting a laboratory into a valid ROSE-T input, the user can simply provide the Kathará scenario and specify the candidate Autonomous System together with its external relationships. The adapter automatically analyses the laboratory and generates all the artifacts required by the verification framework.

In particular, the tool extracts router configurations, generates the candidate description, produces the startup files required by the emulated environment, creates the Global Information File introduced in Section 5.2, and builds a routing table dump consistent with the topology under examination. The resulting directory can therefore be directly consumed by ROSE-T without requiring any additional manual intervention.

Beyond simple conversion, ROSET-Adapter also provides a number of optional transformations aimed at supporting experimentation. For example, the tool can automatically generate startup commands enabling Reverse Path Forwarding (RPF) on candidate routers or inject routing policies intended to prevent route leaks. These mechanisms are not presented as universally correct solutions and do not guarantee MANRS compliance in every scenario. Nevertheless, they provide a convenient starting point for exploratory testing and allow researchers to rapidly evaluate the impact of common mitigation techniques without manually modifying every router configuration.

From the user's perspective, interacting with ROSET-Adapter is intentionally straightforward. The only required inputs are the directory containing the Kathará laboratory, the Autonomous System that should be considered as the candidate network, and the relationships associated with its external neighbours. Once these parameters are provided through the command-line interface, the tool automatically generates a complete ROSE-T workspace within a few seconds.

The structure and contents of the generated workspace will be discussed in

the next section, where the complete execution workflow of the framework is presented.

5.5 User interaction

After introducing the extensions developed in this work and the supporting role of ROSET-Adapter, it is useful to describe the complete workflow required to execute a verification with ROSE-T.

The framework is operated through a command-line interface and requires a working directory containing all the information necessary to reconstruct the candidate Autonomous System and its surrounding environment. While these files can be prepared manually, in practice they are automatically generated by ROSET-Adapter when starting from a Kathará laboratory.

A typical generated workspace is composed of the following files:

```
.  
|-- candidate.json  
|-- r1.frr  
|-- r1_startup.txt  
|-- r2.frr  
|-- r2_startup.txt  
|-- relationships.json  
|-- rib.db  
|-- rib.mrt
```

The central element of the workspace is the `candidate.json` file, which describes the Autonomous System under evaluation and provides all the information required by ROSE-T to reconstruct the emulated topology.

An example is shown below:

```
{
```



```
"local_as": 3,
"relationships": "relationships.json",
"rib_dump": "rib.db",
"routers": [
  {
    "name": "r1",
    "vendor": "frr",
    "config_path": "r1.frr",
    "startup_script_path": "r1_startup.txt"
  },
  {
    "name": "r2",
    "vendor": "frr",
    "config_path": "r2.frr",
    "startup_script_path": "r2_startup.txt"
  }
]
```

The `local_as` field identifies the candidate Autonomous System, while the `routers` section specifies all routers that belong to it. For each router, the framework requires the vendor type, the path to the configuration file, and the path to the corresponding startup script. The `relationships` field points to the Global Information File, whereas `rib_dump` identifies the routing table dump used during topology reconstruction.

Once the workspace has been prepared, executing a validation requires only the candidate description file. ROSE-T automatically loads the referenced artifacts, reconstructs the topology, deploys the laboratory through Kathará, executes the selected MANRS actions, and reports the obtained results.

The framework normally prints its output directly to the terminal. However,

for reproducibility and post-analysis purposes, the generated report can also be written to a file using the `--file` command-line option:

```
python3 test_as_candidate.py candidate.json --file results.txt
```

This functionality proved particularly useful during the experimental campaign presented in Chapter 6, where the results of multiple executions needed to be collected and compared systematically.

Particular attention should be devoted to the routing table dump files. ROSE-T relies on routing information obtained from public BGP collectors in order to reconstruct the external environment surrounding the candidate Autonomous System. Such data is typically distributed in MRT format, represented in the generated workspace by the `rib.mrt` file.

Since repeatedly parsing large MRT dumps would significantly increase execution times, ROSE-T preprocesses the original dump and converts it into an internal database representation, stored in the `rib.db` file. As described in the official ROSE-T documentation, this preprocessing stage extracts and indexes the routing information required by the framework, allowing subsequent executions to access routing entries efficiently without repeatedly parsing the raw MRT data.

As a result, the `rib.mrt` file can be regarded as the original routing information source, while `rib.db` represents an optimized and indexed version specifically designed to accelerate topology reconstruction and validation activities.

Interpreting Results Each validation action can produce three different outcomes:

- **SUCCESS:** the validation completed successfully and no violation was detected.
- **ERROR:** the validation completed successfully and identified a configuration that violates the requirements checked by the corresponding MANRS action.

- **WARNING:** the validation could not be executed reliably because the conditions required by the experiment were not met.

A warning must not be interpreted as either compliance or non-compliance. Rather, it indicates that the framework was unable to construct a testing environment that guarantees the validity of the result. This situation may occur, for example, when the Anti-Spoofing action cannot identify a suitable attachment point for the testing client or when the information required by a validation is unavailable.

At the end of the execution, ROSE-T aggregates the outcomes produced by all actions and reports them to the user. An Autonomous System can be considered compliant with respect to the validations implemented by ROSE-T only when all executed actions complete successfully. The presence of at least one error indicates that a violation has been detected, whereas warnings require additional investigation since the corresponding validation was not completed.

5.6 Conclusions and comments

The extensions presented throughout this chapter substantially broaden the applicability of ROSE-T while preserving its original design principles. Rather than redesigning the framework from scratch, the proposed modifications build upon its modular architecture, extending the existing components to support more realistic network environments and experimental workflows.

As illustrated in Figure 5.3, the overall software architecture remains largely unchanged. The modular organization based on independent components and the use of established design patterns, such as the Adapter, Factory and Singleton patterns, continue to provide a flexible and extensible foundation. Most of the changes are therefore concentrated within individual modules rather than in the interactions between them.

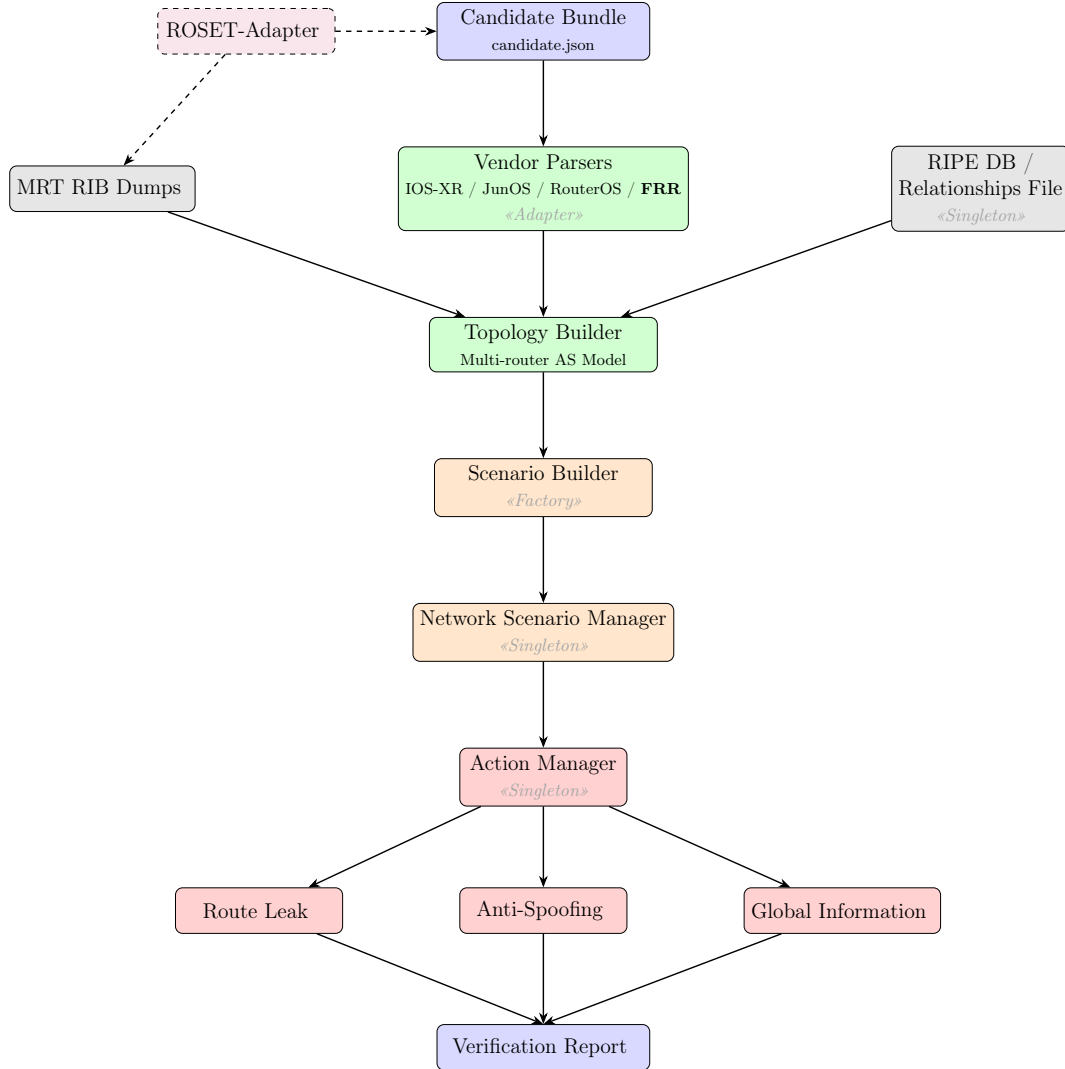


Figure 5.3: Architecture of the extended ROSE-T framework.

The most significant architectural evolution concerns the representation of the candidate Autonomous System. Instead of assuming a single border router, the internal topology model has been redesigned to describe an arbitrary number of routers belonging to the same AS, together with their internal connectivity and routing relationships. This richer representation enables the framework to emulate significantly more realistic deployments while remaining fully compatible with the original verification workflow.

The parsing subsystem has been extended through the introduction of native FRRouting support, allowing laboratories entirely based on open-source routing software to be analysed without requiring configuration translation. In parallel, the introduction of the Global Information File allows routing relationships and registry information to be supplied locally, enabling fully controlled experiments independently of external Internet routing registries. Finally, the ROSET-Adapter complements the framework by automatically generating all the artifacts required by ROSE-T starting from a standard Kathará laboratory, considerably simplifying the preparation of experimental scenarios.

Although the internal data structures have evolved considerably, the verification pipeline itself remains conceptually unchanged. The framework still follows the same sequence of operations: the candidate input is parsed, the surrounding network topology is reconstructed, an emulated network scenario is generated, routing convergence is verified, and the MANRS verification actions are executed before producing the final report. The difference lies in the scope and richness of the processed information. The input is no longer limited to a single router configuration but consists of a complete candidate bundle describing an entire Autonomous System, while the topology reconstruction and scenario generation phases now operate on multiple interconnected routers. Similarly, the verification actions have been redesigned to evaluate the behaviour of the candidate AS as a whole rather than that of a single border router, resulting in a more representative assessment of routing security in realistic operational environments.

Overall, the proposed extensions preserve the philosophy of the original framework while removing several of its most significant practical limitations. The resulting system supports a wider range of routing platforms, enables reproducible laboratory experimentation, and provides a verification environment capable of representing modern Autonomous Systems with multiple routers without sacrificing automation or extensibility.

Chapter 6

Tests and Results

6.1 Evaluation Methodology

This chapter presents the experimental evaluation of the extensions developed for ROSE-T, covering both the MANRS compliance verification workflow and the topology reconstruction capabilities introduced in the previous chapters.

The evaluation is organized into two complementary categories of scenarios, each designed to assess a different aspect of the proposed system.

The first category consists of a set of controlled scenarios specifically designed to evaluate ROSE-T’s ability to correctly assess MANRS compliance under well-defined conditions. Each scenario is associated with a manually established ground truth that specifies the expected outcome for every MANRS Action, expressed as `SUCCESS`, `WARNING`, or `ERROR`. The ground truth is obtained through manual inspection of the network topology, routing configuration and expected routing behaviour before executing any automated verification.

The controlled scenarios include networks exhibiting different levels of compliance, ranging from intentionally misconfigured deployments to fully compliant configurations. For each non-compliant scenario, a corresponding compliant version is also provided. These paired scenarios allow us to verify not only that

ROSE-T correctly identifies compliance violations, but also that the same violations disappear once the network configuration is corrected. This provides additional confidence in the correctness and consistency of the verification process.

The second category consists of realistic network scenarios based on existing Kathará laboratories publicly available on the Kathará repository [Fra26]. Unlike the controlled scenarios, these laboratories were not originally designed for MANRS compliance evaluation. Instead, they are used to assess the new topology reconstruction capabilities introduced in ROSE-T. ROSET-Adapter plays a crucial role in this phase because, thanks to it, is possible to test scenarios not originally built for ROSE-T.

For these scenarios, ROSET-Adapter is used to extract routing information from the router configurations and generate the MRT RIB dumps required by ROSE-T. This step is intended to emulate the workflow adopted during a real MANRS assessment, where routing information is typically obtained from public route collectors and provided to the verification engine in the form of MRT RIB dumps. As discussed in previous chapters, ROSE-T can directly operate on real routing data collected from operational networks. However, since the scenarios considered in this evaluation are synthetic laboratories executed entirely within Kathará, obtaining routing information from external collectors would not be meaningful. The use of ROSET-Adapter therefore provides a controlled mechanism for generating the routing data required by the verification process while preserving the same input format and processing pipeline used in real-world deployments.

Since the original laboratories were not designed with compliance validation in mind, the focus of this evaluation shifts from the correctness of the compliance assessment to the quality of the reconstructed topology and the robustness of the overall extraction and verification pipeline.

Additionally, ROSET-Adapter provides mechanisms that can assist in the

generation of compliant variants of existing scenarios by automatically applying a set of configuration transformations. Although this functionality is not intended to be a primary feature of the framework and cannot guarantee a successful outcome for every possible network configuration, it provides a useful guideline for exploring remediation strategies and evaluating the effect of configuration changes on compliance.

For each scenario, the evaluation includes a description of the network topology and routing configuration, followed by the results produced by ROSE-T. In the controlled scenarios, these results are explicitly compared against the predefined ground truth. In the other scenarios, the discussion focuses on the fidelity of the reconstructed topology, the correctness of the extracted routing information and any limitations or issues encountered during the reconstruction process, for a deeper analysis of how those experiments were evaluated, see Section 6.3.1

6.2 Controlled Scenarios

The controlled scenarios constitute the first part of the experimental evaluation and are specifically designed to assess the correctness of the verification process. Unlike the benchmark laboratories presented later, these scenarios were purpose-built to exercise individual MANRS actions under controlled conditions and to provide a well-defined ground truth against which the results produced by ROSE-T can be compared.

Each laboratory intentionally contains one or more routing misconfiguration affecting different MANRS requirements. Prior to the experiments, the expected outcome of every verification action was manually determined through inspection of the network topology and router configurations. This ground truth provides the reference used to evaluate the correctness of the framework.

To further validate the proposed approach, a second version of every laboratory was also developed. Starting from the intentionally misconfigured scenario,

the routing policies and security mechanisms responsible for the detected violations were corrected and the complete verification pipeline was executed again. The objective of these additional experiments is not to identify the only possible configuration capable of satisfying the MANRS requirements, but rather to demonstrate that the reported violations correspond to actual configuration issues and disappear once appropriate corrective measures are applied. Consequently, each controlled scenario is evaluated twice: first in its original configuration and then after the identified issues have been addressed.

The selected laboratories were designed with increasing structural complexity in order to progressively challenge both ROSE-T and ROSET-Adapter. While all three scenarios feature a candidate Autonomous System interconnected with neighbouring networks through BGP, each introduces additional topological characteristics that make topology reconstruction and policy verification progressively more demanding. Goldfish represents the simplest scenario, Pram introduces multiple customer relationships connected to the same border router and Shuttle models a distributed multi-router Autonomous System with multi-homed customers and internal iBGP sessions.

6.2.1 Goldfish

Goldfish, represented in the Figure 6.1, is the simplest scenario considered in the evaluation and serves as a baseline for validating the correctness of the extraction and reconstruction pipeline. The candidate network is AS101, which consists of multiple routers connected through a relatively simple internal topology. Internal routing information is exchanged through directly connected iBGP sessions, making the internal structure straightforward to infer.

From an inter-domain perspective, AS101 is connected to three neighbouring Autonomous Systems implementing the three most common business relationships encountered on the Internet: a provider, a customer and a peer. This combination makes Goldfish particularly suitable for evaluating whether the framework can correctly identify and reconstruct different types of AS relationships while operating on a topology whose internal structure remains relatively easy to analyse.

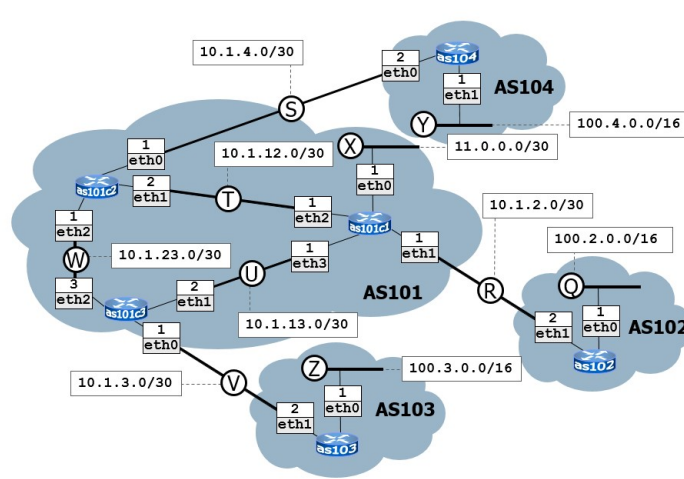


Figure 6.1: Goldfish Scenario.

In the Goldfish scenario, the information published in the routing registry is consistent with the routing relationships observed within the network. Conse-

quently, the *Global Information* verification is expected to succeed.

The candidate Autonomous System is also configured with the *anti-spoofing* protections required by the verification process, leading to a positive outcome for the corresponding check.

On the other hand, the routing policies have been intentionally configured to allow a route-leak condition. Therefore, the *Route Leak* verification is expected to detect a violation and report a negative result.

Table 6.1 summarizes the expected results.

Verification Check	Expected Result
Route Leak	ERROR
Global Information	SUCCESS
Anti-Spoofing	SUCCESS

Table 6.1: Expected results for the Goldfish scenario.

6.2.1.1 Goldfish Results

Table 6.2 reports the outcome of the validation checks performed on the Goldfish scenario. Overall, the generated configuration behaves as expected. The Route Leak check correctly reports an ERROR, consistently with the intended policy violation injected in the scenario, while the Global Information check completes successfully, confirming that the routing information reconstructed by the framework matches the routing policies declared in the RIR database.

The Anti-Spoofing validation reports a WARNING instead of a complete success. As anticipated in the previous chapter, the anti-spoofing verification requires the automatic deployment of client hosts connected to suitable interfaces of the candidate AS. The client placement algorithm searches for interfaces satisfying a number of topological constraints to ensure that the generated traffic effectively traverses the candidate router and reaches the provider under test. In practice, not every topology contains interfaces satisfying these constraints.

In the Goldfish scenario one of the candidate router interfaces does not allow the deployment of a suitable client, preventing the execution of the corresponding anti-spoofing test. This behaviour is reflected in the execution log visible in Figure 6.2, which reports:

No client available for as101_c3, cannot perform anti-spoofing check.

Therefore, the reported warning does not indicate an incorrect anti-spoofing configuration, but rather that one of the validation instances could not be executed because the required network conditions were not present. The remaining anti-spoofing checks complete successfully, demonstrating that the generated filtering rules correctly prevent packets with spoofed source addresses from reaching the provider.

Finally, the Route Leak check correctly identifies that the test prefix is advertised towards the provider, exactly as expected for the intentionally misconfigured scenario. This confirms that the framework is able not only to verify correct configurations, but also to detect routing policy violations introduced in the laboratory.

```

WARNING | No networks announced in IPv6, skipping...
SUCCESS | [Global Information] Excellent work, what declared in the RIR DB matches what was found on the Internet (and vice versa)!
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 155.86.128.0/17 towards provider AS104 via as101_c1. The packet transmitted was SrcIP=155.86.128.2 -> DstIP=100.4.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 155.86.128.0/17 towards provider AS104 via as101_c2. The packet transmitted was SrcIP=155.86.128.2 -> DstIP=100.4.0.2.
WARNING | [Anti-Spoofing Check] No client available for as101_c3, cannot perform anti-spoofing check.
WARNING | [Anti-Spoofing Check] No networks announced in IPv6.
ERROR   | [Route Leak Check] Configuration allows to announce the spoofed network 87.238.128.0/17 of customer AS103 towards provider AS104 via as101_c2.
WARNING | [Route Leak Check] No networks announced in IPv6.

```

Figure 6.2: Verification output for the Goldfish scenario.

Check	Expected	Observed
Route Leak	ERROR	ERROR
Global Information	SUCCESS	SUCCESS
Anti-Spoofing	SUCCESS	WARNING

Table 6.2: Expected versus observed results for the Goldfish scenario.

Table 6.3 reports the execution time of each validation phase. The overall execution requires approximately 69 seconds, with the Anti-Spoofing validation accounting for almost the entire running time.

This behaviour is expected because, unlike the other validation steps, the anti-spoofing check requires the deployment of test clients, the generation of active traffic, packet capture, timeout management and the execution of multiple validation attempts for each combination provider-candidate router. While the Route Leak check also performs active validation by injecting BGP announcements and observing their propagation, its execution time remains significantly lower than that of the anti-spoofing check. The difference is mainly due to the number of validation instances: the Route Leak validation requires a single announcement test per customer of the candidate AS against each provider; moreover, the timeout adopted in this case is significantly shorter than the one used for the anti-spoofing check. Consequently, the overall execution time is substantially reduced.

Finally, these results indicate that the framework is able to perform a complete validation of a laboratory scenario in approximately one minute, making it suitable both for automated testing during scenario development and for integration into continuous validation workflows.

Phase	Duration
Build topology	0.00s
Global Information	0.29s
Anti-Spoofing Check	66.53s
Route Leak Check	2.49s
Undeploy	0.00s
Total	69.31

Table 6.3: Execution times for the Goldfish scenario.

6.2.2 Goldfish MANRS compliant

The original Goldfish scenario intentionally contained a route leak violation, which was correctly detected by the validation framework. To obtain a compliant configuration, an outbound route-map was introduced on the eBGP session towards the upstream provider. The route-map permits only announcements cor-

responding to prefixes that legitimately belong to the local Autonomous System and to its customer cone, while denying all other advertisements.

Ideally, the list of authorized prefixes should be automatically generated from Internet Routing Registry (IRR) information. Since the laboratory topology used in this work is entirely synthetic and under our control, the legitimate prefixes are known a priori. Therefore, the prefix list was manually constructed to include only the prefixes belonging to the local AS and its legitimate customers, while all unauthorized announcements were filtered.

As shown in Figure 6.3, after applying the outbound filtering policy, the Route Leak validation completed successfully, confirming that only legitimate prefixes were propagated towards upstream providers.

```
WARNING | No networks announced in IPv6, skipping...
SUCCESS | [Global Information] Excellent work, what declared in the RIR DB matches what was found on the Internet (and vice versa)!
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 108.69.0.0/17 towards provider AS104 via as101_c1. The packet transmitted was SrcIP=108.69.0.2 -> DstIP=100.4.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 108.69.0.0/17 towards provider AS104 via as101_c2. The packet transmitted was SrcIP=108.69.0.2 -> DstIP=100.4.0.2.
WARNING | [Anti-Spoofing Check] No client available for as101_c3, cannot perform anti-spoofing check.
WARNING | [Anti-Spoofing Check] No networks announced in IPv6.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 82.142.128.0/17 of customer AS103 towards provider AS104 via as101_c2.
WARNING | [Route Leak Check] No networks announced in IPv6.
```

Figure 6.3: Output Goldfish MANRS compliant.

6.2.3 Pram

Pram, represented in the Figure 6.4, introduces a higher level of complexity while preserving a relatively compact topology. The candidate network is AS20, whose routers maintain directly connected iBGP sessions similarly to the previous scenario. However, the external connectivity differs significantly from Goldfish.

The main characteristic of this laboratory is the presence of multiple customer networks connected through the same border router. This configuration was specifically selected because it represents a common operational scenario that cannot be fully captured by simpler topologies where each external relationship is associated with a dedicated edge router.

As a result, Pram provides a useful test case for evaluating whether the reconstruction process can correctly infer multiple independent customer relationships originating from the same point of attachment to the candidate Autonomous System. Compared to Goldfish, the scenario requires the framework to distinguish a more accurate set of inter-domain relationships while preserving the correctness of the internal topology reconstruction.

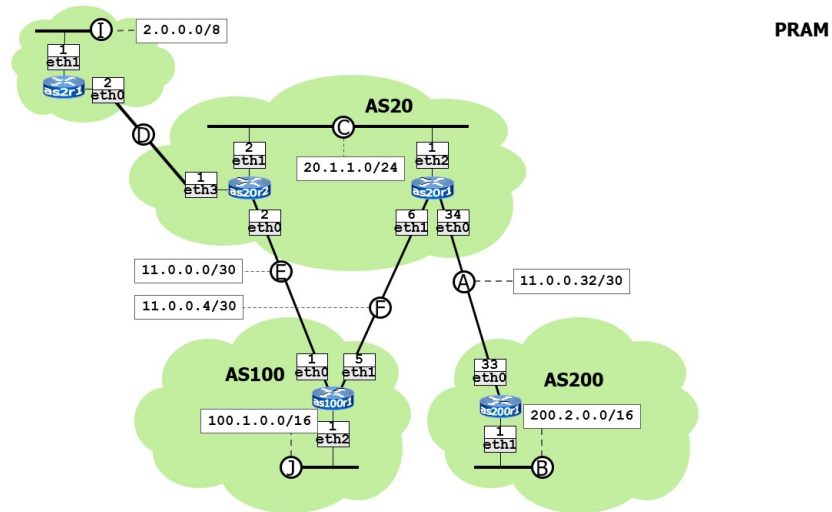


Figure 6.4: Pram Scenario.

The expected results for the Pram scenario are summarized in Table 6.4. The candidate network was intentionally configured so that all transit relationships are correctly reflected in the routing registry information and no route-leak conditions are present. Therefore, both the *Global Information* and *Route Leak* checks are expected to succeed.

However, no anti-spoofing mechanisms have been deployed on the candidate Autonomous System. As a consequence, the corresponding verification is expected to report a compliance violation.

Verification Check	Expected Result
Route Leak	SUCCESS
Global Information	SUCCESS
Anti-Spoofing	ERROR

Table 6.4: Expected results for the Pram scenario.

6.2.3.1 Pram Results

Table 6.5 summarizes the validation results obtained for the Pram scenario. As in the previous case, the Global Information check completes successfully, confirming that the routing information reconstructed from the emulated topology is consistent with the information declared in the RIR database. Likewise, the Route Leak check produces the expected outcome by correctly identifying the intentionally injected routing policy violation.

Unlike the Goldfish scenario, the Anti-Spoofing validation completes successfully without reporting any warnings. This indicates that the client placement algorithm was able to identify suitable attachment points for every candidate router, allowing all the planned validation tests to be executed. Consequently, every anti-spoofing policy was effectively verified and the generated filtering rules correctly prevented packets with spoofed source addresses from reaching the provider networks.

The execution log in Figure 6.5 confirms that spoofed packets originating from the candidate network are consistently blocked before leaving the AS, while the Route Leak validation successfully verifies that announcements of test prefixes are filtered and never propagated towards upstream providers. Overall, the results demonstrate that the generated configuration satisfies all the expected security requirements for this scenario.

```

WARNING | No networks announced in IPv6, skipping...
SUCCESS | [Global Information] Excellent work, what declared in the RIR DB matches what was found on the Internet (and vice versa)!
ERROR   | [Anti-Spoofing Check] Configuration allows to send a spoofed packet from network 126.192.0.0/10 towards provider AS2 via as20_r1. The packet transmitted was SrcIP=126.192.0.2 -> DstIP=2.0.0.2.
ERROR   | [Anti-Spoofing Check] Configuration allows to send a spoofed packet from network 126.192.0.0/10 towards provider AS2 via as20_r2. The packet transmitted was SrcIP=126.192.0.2 -> DstIP=2.0.0.2.
WARNING | [Anti-Spoofing Check] No networks announced in IPv6.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 140.214.0.0/15 of customer AS200 towards provider AS2 via as20_r2.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 140.214.0.0/15 of customer AS100 towards provider AS2 via as20_r2.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 140.214.0.0/15 of customer AS100 towards provider AS2 via as20_r2.
WARNING | [Route Leak Check] No networks announced in IPv6.

```

Figure 6.5: Verification output for the Pram scenario.

Check	Expected	Observed
Route Leak	SUCCESS	SUCCESS
Global Information	SUCCESS	SUCCESS
Anti-Spoofing	ERROR	ERROR

Table 6.5: Expected versus observed results for the Pram scenario.

Table 6.6 reports the execution times of the different validation phases. The complete validation requires approximately 70 seconds, a value comparable to that observed for the Goldfish scenario. As expected, the Anti-Spoofing check represents the most time-consuming phase, accounting for nearly one minute of the total execution time.

Overall, the execution times confirm that the framework is capable of performing a complete validation of medium-sized scenarios in approximately one minute, while successfully executing every planned validation test without requiring manual intervention.

6.2.4 Pram MANRS compliant

To obtain a fully compliant anti-spoofing configuration, Unicast Reverse Path Forwarding (uRPF) was enabled on all interfaces of the routers belonging to the

Phase	Duration
Build topology	0.00s
Global Information	0.34s
Anti-Spoofing Check	59.47s
Route Leak Check	7.13s
Undeploy	0.00s
Total	69.94

Table 6.6: Execution times for the Pram scenario.

candidate Autonomous System.

Reverse Path Forwarding validates the source address of every incoming packet by verifying that the routing table contains a valid reverse path through the same interface on which the packet was received. If no such route exists, the packet is discarded before entering the forwarding process. This mechanism effectively prevents hosts from transmitting packets with forged source addresses outside the Autonomous System, enforcing source address validation at the network edge.

Once uRPF was enabled throughout the candidate ASes, all anti-spoofing validation tests completed successfully. The framework confirmed that spoofed packets were consistently discarded before reaching upstream providers, demonstrating the effectiveness of the applied filtering mechanism.

Figure 6.6 shows the results of the execution.

```

WARNING | No networks announced in IPv6, skipping...
SUCCESS | [Global Information] Excellent work, what declared in the RIR DB matches what was found on the Internet (and vice versa)!
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 157.192.0.0/10 towards provider AS2 via as20_r1. The packet transmitted was SrcIP=157.192.0.2 -> DstIP=2.0.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 157.192.0.0/10 towards provider AS2 via as20_r2. The packet transmitted was SrcIP=157.192.0.2 -> DstIP=2.0.0.2.
WARNING | [Anti-Spoofing Check] No networks announced in IPv6.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 29.195.128.0/17 of customer AS200 towards provider AS2 via as20_r2.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 29.195.128.0/17 of customer AS100 towards provider AS2 via as20_r2.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 29.195.128.0/17 of customer AS100 towards provider AS2 via as20_r2.
WARNING | [Route Leak Check] No networks announced in IPv6.

```

Figure 6.6: Output Pram MANRS compliant.

6.2.5 Shuttle

The *Shuttle Shuttle*, represented in the Figure 6.7, is the most complex scenario considered in the evaluation. The candidate network is AS1, which contains multiple routers interconnected through a non-trivial internal topology. Unlike the previous laboratories, the routers participating in the candidate Autonomous System are not all directly connected through iBGP sessions, resulting in a more realistic internal routing structure and increasing the difficulty of topology inference.

From an inter-domain perspective, the scenario includes multiple external networks connected through different border routers. In particular, AS1 maintains connectivity with multiple customer Autonomous Systems and two different providers leading to a multi-homed¹ scenario. Such configurations are frequently encountered in operational networks and introduce additional complexity in the interpretation of routing information and AS relationships.

The combination of a larger internal topology, multiple border routers and multi-homed external networks makes Shuttle an effective stress test for the proposed reconstruction methodology. The scenario allows the evaluation of the framework under conditions that more closely resemble real-world deployments, where routing information may originate from multiple interconnected sources and where relationship inference is significantly more challenging.

The Shuttle scenario represents the most challenging configuration among the selected laboratories and contains multiple intentional compliance violations.

The routing policies of the candidate Autonomous System allow route advertisements that generate a *route-leak* condition, which should be detected by the corresponding verification procedure. Furthermore, the transit relationships declared through the routing registry information do not match those observed

¹Multi-homing is the practice of connecting a host or network to more than one network or Internet Service Provider (ISP) simultaneously to increase reliability, performance and security. By maintaining multiple active connections, a multi-homed system can automatically reroute traffic if one link fails, ensuring uninterrupted connectivity and preventing downtime.

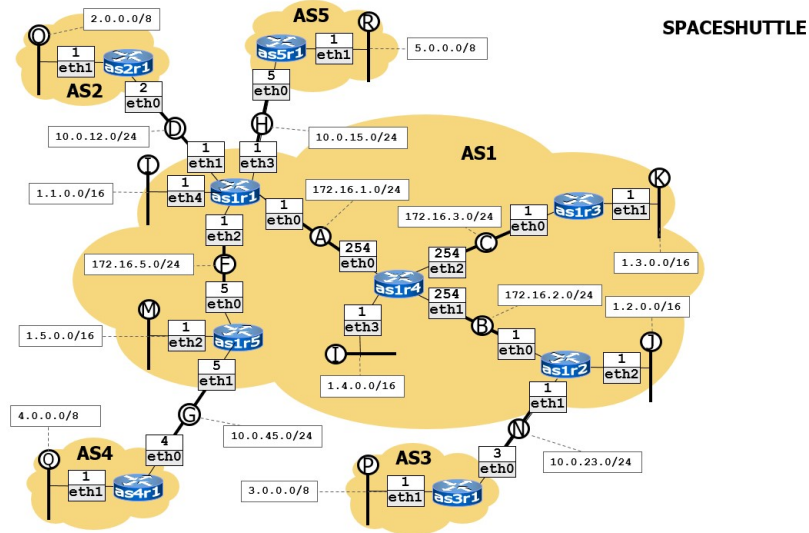


Figure 6.7: Shuttle Scenario.

in the routing data, causing the *Global Information* verification to fail.

Finally, *anti-spoofing* protections are not deployed within the candidate network. As a result, all considered verification checks are expected to report non-compliant behaviour, as clearly shown in Table 6.7.

Verification Check	Expected Result
Route Leak	ERROR
Global Information	ERROR
Anti-Spoofing	ERROR

Table 6.7: Expected results for the Shuttle scenario.

6.2.5.1 Shuttle Results

Table 6.8 summarizes the validation results obtained for the Shuttle scenario, visible in Figure 6.8. Unlike the previous case studies, every validation check reports an ERROR, exactly as expected from the intentionally misconfigured topology. This scenario was specifically designed to include multiple policy violations si-

multaneously, providing a comprehensive evaluation of the framework’s ability to identify different classes of routing and security misconfiguration within a single experiment.

The Global Information validation correctly detects inconsistencies between the routing information reconstructed from the emulated topology and the data extracted from the RIR database, confirming that the announced prefixes do not match the expected Internet routing information.

Similarly, the Anti-Spoofing validation reports failures for all candidate routers under test. The generated configurations allow packets with spoofed source addresses to be forwarded towards upstream providers, demonstrating the absence or incorrect implementation of source address validation mechanisms. The framework successfully identifies these violations by actively generating spoofed traffic and verifying that it reaches the provider network, producing the expected error reports.

Finally, the Route Leak validation also behaves as expected by detecting that customer prefixes are incorrectly propagated towards upstream providers. Rather than filtering unauthorized announcements, the affected routers advertise prefixes that should remain confined to customer relationships, resulting in the expected route leak errors.

Overall, this scenario demonstrates that the proposed validation framework is capable of identifying multiple independent classes of configuration errors within the same laboratory, while correctly distinguishing them according to the corresponding validation category.

```

WARNING | No networks announced in IPv6, skipping...
ERROR   | [(Global Information) The following transits for AS1 are declared in the RIR DB but not detected on the Internet: 100
ERROR   | [(Anti-Spoofing Check) Configuration allows to send a spoofed packet from network 77.172.0.0/17 towards provider AS2 via as1_r1. The packet transmitted was SrcIP=77.172.0.2 -> DstIP=2.0.0.2.
ERROR   | [(Anti-Spoofing Check) Configuration allows to send a spoofed packet from network 77.172.0.0/17 towards provider AS2 via as1_r2. The packet transmitted was SrcIP=77.172.0.2 -> DstIP=2.0.0.2.
ERROR   | [(Anti-Spoofing Check) Configuration allows to send a spoofed packet from network 77.172.0.0/17 towards provider AS2 via as1_r3. The packet transmitted was SrcIP=77.172.0.2 -> DstIP=2.0.0.2.
ERROR   | [(Anti-Spoofing Check) Configuration allows to send a spoofed packet from network 77.172.0.0/17 towards provider AS2 via as1_r5. The packet transmitted was SrcIP=77.172.0.2 -> DstIP=2.0.0.2.
ERROR   | [(Anti-Spoofing Check) Configuration allows to send a spoofed packet from network 77.172.0.0/17 towards provider AS5 via as1_r1. The packet transmitted was SrcIP=77.172.0.2 -> DstIP=5.0.0.2.
ERROR   | [(Anti-Spoofing Check) Configuration allows to send a spoofed packet from network 77.172.0.0/17 towards provider AS5 via as1_r2. The packet transmitted was SrcIP=77.172.0.2 -> DstIP=5.0.0.2.
ERROR   | [(Anti-Spoofing Check) Configuration allows to send a spoofed packet from network 77.172.0.0/17 towards provider AS5 via as1_r3. The packet transmitted was SrcIP=77.172.0.2 -> DstIP=5.0.0.2.
ERROR   | [(Anti-Spoofing Check) Configuration allows to send a spoofed packet from network 77.172.0.0/17 towards provider AS5 via as1_r5. The packet transmitted was SrcIP=77.172.0.2 -> DstIP=5.0.0.2.
WARNING | [(Anti-Spoofing Check) No networks announced in IPv6.
ERROR   | [(Route Leak Check) Configuration allows to announce the spoofed network 94.246.0.0/17 of customer AS3 towards provider AS2 via as1_r1.
ERROR   | [(Route Leak Check) Configuration allows to announce the spoofed network 94.246.0.0/17 of customer AS3 towards provider AS5 via as1_r1.
ERROR   | [(Route Leak Check) Configuration allows to announce the spoofed network 94.246.0.0/17 of customer AS4 towards provider AS2 via as1_r1.
ERROR   | [(Route Leak Check) Configuration allows to announce the spoofed network 94.246.0.0/17 of customer AS4 towards provider AS5 via as1_r1.
WARNING | [(Route Leak Check) No networks announced in IPv6.

```

Figure 6.8: Verification output for the Shuttle scenario.

Check	Expected	Observed
Route Leak	ERROR	ERROR
Global Information	ERROR	ERROR
Anti-Spoofing	ERROR	ERROR

Table 6.8: Expected versus observed results for the Shuttle scenario.

Table 6.9 reports the execution times for the Shuttle scenario. The complete validation requires approximately 247 seconds, making it significantly more expensive than the previous experiments.

The increase in execution time is almost entirely attributable to the Anti-Spoofing validation, which requires over four minutes to complete. This is expected, as the Shuttle topology contains a considerably larger number of candidate routers and provider relationships than the previous scenarios. Since the anti-spoofing validation actively tests every candidate router against each of its provider connections, the total number of client deployments, traffic generation steps and packet capture operations grows proportionally with the size and connectivity of the topology.

The execution time of the Global Information and Route Leak checks remains comparatively low. Although the Route Leak validation also relies on active BGP announcements, it performs a substantially smaller number of experiments than the Anti-Spoofing validation: only 4 tests, compared to the 8 performed by the Anti-Spoofing check.

Despite the increased execution time, the results show that the framework scales to more complex laboratory scenarios while successfully detecting every intentionally introduced misconfiguration. This confirms that the proposed validation methodology remains effective even when several routing and security policy violations coexist within the same network.

Phase	Duration
Build topology	0.00s
Global Information	0.32s
Anti-Spoofing Check	241.95s
Route Leak Check	5.14s
Undeploy	0.00s
Total	247.41

Table 6.9: Execution times for the Shuttle scenario.

6.2.6 Shuttle MANRS compliant

The Shuttle scenario intentionally combined multiple classes of misconfiguration. Consequently, obtaining a compliant configuration required modifications addressing every validation category.

The Route Leak violations were resolved by introducing outbound filtering policies on provider-facing BGP sessions, similarly to the approach adopted for the Goldfish scenario. The configured route-maps restrict advertisements to prefixes that are legitimately owned by the local Autonomous System or by its customer cone.

The Anti-Spoofing violations were corrected by enabling uRPF over the interfaces of all the candidate routers, preventing packets with forged source addresses from being forwarded towards upstream providers.

Finally, the Global Information validation originally failed because the local dataset used to emulate the information collected from Internet Routing Registries intentionally contained a transit Autonomous System that did not actually exist in the routing information reconstructed from the BGP announcements. As a consequence, the validation framework correctly identified an inconsistency between the registry information and the routing data obtained from the simulated Internet dump.

To eliminate this inconsistency, the fictitious transit AS was removed from the local registry dataset. This restored consistency between the registry information

and the BGP announcements generated through the ROSET-Adapter, allowing the Global Information validation to complete successfully.

Following these modifications, all validation checks completed successfully, as shown in Figure 6.9, demonstrating that the framework is able not only to detect routing and security policy violations, but also to verify that the applied configuration changes effectively restore compliance with the expected Internet routing policies.

```

WARNING | No networks announced in IPv6, skipping...
SUCCESS | [Global Information] Excellent work, what declared in the RIR DB matches what was found on the Internet (and vice versa)!
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 63.217.128.0/17 towards provider AS2 via as1_r1. The packet transmitted was SrcIP=63.217.128.2 -> DstIP=2.0.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 63.217.128.0/17 towards provider AS2 via as1_r2. The packet transmitted was SrcIP=63.217.128.2 -> DstIP=2.0.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 63.217.128.0/17 towards provider AS2 via as1_r3. The packet transmitted was SrcIP=63.217.128.2 -> DstIP=2.0.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 63.217.128.0/17 towards provider AS2 via as1_r5. The packet transmitted was SrcIP=63.217.128.2 -> DstIP=2.0.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 63.217.128.0/17 towards provider AS5 via as1_r1. The packet transmitted was SrcIP=63.217.128.2 -> DstIP=5.0.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 63.217.128.0/17 towards provider AS5 via as1_r2. The packet transmitted was SrcIP=63.217.128.2 -> DstIP=5.0.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 63.217.128.0/17 towards provider AS5 via as1_r3. The packet transmitted was SrcIP=63.217.128.2 -> DstIP=5.0.0.2.
SUCCESS | [Anti-Spoofing Check] Configuration correctly blocks a spoofed packet from network 63.217.128.0/17 towards provider AS5 via as1_r5. The packet transmitted was SrcIP=63.217.128.2 -> DstIP=5.0.0.2.
WARNING | [Anti-Spoofing Check] No networks announced in IPv6.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 81.208.0.0/17 of customer AS3 towards provider AS2 via as1_r1.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 81.208.0.0/17 of customer AS3 towards provider AS5 via as1_r1.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 81.208.0.0/17 of customer AS4 towards provider AS2 via as1_r1.
SUCCESS | [Route Leak Check] Configuration correctly blocks announcements of the spoofed network 81.208.0.0/17 of customer AS4 towards provider AS5 via as1_r1.
WARNING | [Route Leak Check] No networks announced in IPv6.

```

Figure 6.9: Output Shuttle MANRS compliant.

6.3 Benchmark Scenarios

In addition to the laboratories selected from Kathará repository, a second set of experiments is conducted using the same network topologies but selecting different Autonomous Systems as candidates for the verification process. In particular, several networks that originally appeared as customer ASes in the controlled scenarios are subsequently evaluated as candidate Autonomous Systems. This approach allows the assessment of the framework under a broader range of network perspectives without requiring the creation of additional laboratory environments.

It is important to note that not every Autonomous System within a topology is suitable as a candidate for verification. The current ROSE-T actions are designed to evaluate routing practices that inherently involve connectivity towards upstream networks, such as transit relationships and route propagation policies. Consequently, Autonomous Systems that do not maintain at least one provider relationship cannot be meaningfully assessed by several verification actions. For

this reason, only Autonomous Systems with at least one upstream provider are selected as candidates during this phase of the evaluation.

This additional set of experiments increases the diversity of the tested configurations while simultaneously providing further validation of the extraction, topology reconstruction and compliance verification pipeline across different positions within the same network topology.

6.3.1 Benchmark Scenarios Evaluation

The controlled experiments presented in the previous sections demonstrate that the proposed framework correctly detects routing policy violations and validates compliant configurations in laboratory scenarios specifically designed for the evaluation. While these experiments allow a precise comparison between expected and observed outcomes, they do not assess the framework's ability to operate on arbitrary topologies that were not originally developed with the proposed validation methodology in mind.

Unlike the controlled scenarios, the expected outcome of every validation is not necessarily known a priori. Consequently, the objective of this evaluation is not to determine whether individual routing policies are correct, but rather to assess whether the framework is able to successfully adapt the laboratory, deploy the validation infrastructure and execute the complete validation pipeline without manual intervention.

For this reason, the evaluation focuses on four complementary metrics, each targeting a different stage of the framework.

Topology Reconstruction The first metric evaluates the ability of the adaptation module to correctly reconstruct the original laboratory topology. For each scenario, the number of routers instantiated by the framework is compared with the number of routers expected from the original laboratory. A successful recon-

struction indicates that the topology extraction process correctly identified the network devices and generated a consistent deployment configuration.

Routing Convergence Successfully deploying the topology is not sufficient to guarantee that the validation can be executed. Therefore, the second metric evaluates whether the generated network reaches a stable routing state after deployment. In particular, the presence of established BGP sessions and the successful convergence of the routing protocol are verified before any validation procedure is executed. Only converged topologies are considered suitable for the subsequent validation phases.

Validation Coverage The third metric measures the percentage of validation tests that are actually executed. As discussed in Section 4.3, certain validation procedures, particularly the Anti-Spoofing validation, require the automatic deployment of client hosts attached to interfaces satisfying a number of topological constraints. Although the client placement algorithm attempts to identify suitable attachment points automatically, some network topologies may not provide interfaces fulfilling all the required conditions. Consequently, a validation may occasionally be skipped despite the topology being otherwise correctly reconstructed.

Rather than evaluating the outcome of individual tests, this metric measures the ratio between successfully executed validations and the total number of validations planned for each laboratory.

Compliant Validation In addition to the original laboratory scenarios, a second version of each laboratory was automatically generated by applying the configuration options introduced in Section 5.4. In particular, the `--policy` option generates outbound BGP filtering policies intended to prevent route leaks, while the `--rpf` option enables Unicast Reverse Path Forwarding (uRPF) on customer-facing interfaces to enforce source address validation.

According to the methodology proposed in this work, these modifications are expected to produce laboratory configurations that comply with the routing policies validated by the framework. Consequently, this second set of experiments provides an opportunity to assess whether the automatically generated configurations effectively satisfy the proposed validation criteria.

It is important to emphasize, however, that these automatically generated configurations should not be regarded as universally correct solutions. Internet routing policies are inherently heterogeneous and operational networks often rely on routing decisions that cannot be inferred solely from publicly available information. As a result, no static policy generation strategy can guarantee compliance in every possible deployment scenario.

Rather than claiming to automatically solve routing policy violations, the proposed approach aims to generate configurations that are consistent with the validation methodology presented in this work and with the routing information available to the framework. Therefore, this metric evaluates the extent to which the automatically adapted laboratories become compliant with the proposed validation model, rather than with the entirety of real-world operational practices.

Laboratory	Topology Reconstruction	BGP Convergence	Validation Coverage	Compliant Validation
Alien-3	7 / 7	✓	4 / 8	×
Alien-4	3 / 3	✓	1 / 1	×
Alien-5	3 / 3	✓	1 / 1	×
Butterfly-50	7 / 7	✓	6 / 7	✓
Butterfly-100	5 / 5	✓	0 / 4	✓
Clothcap-11	7 / 7	✓	5 / 5	×
Galaxy-2	4 / 4	✓	2 / 2	×
Garden-3	6 / 6	✓	6 / 6	✓
Garden-6	4 / 4	✓	1 / 2	×
Redbaron-10	5 / 5	×	0 / 2	×
Shuttle-3	3 / 3	✓	1 / 1	✓
Shuttle-4	3 / 3	✓	1 / 1	✓
T1c2-1	5 / 5	✓	2 / 3	✓
T1c2-4	5 / 5	✓	1 / 2	✓
T1c2-5	3 / 3	✓	0 / 1	×

Table 6.10: Summary of the evaluation performed on non-controlled laboratory scenarios.

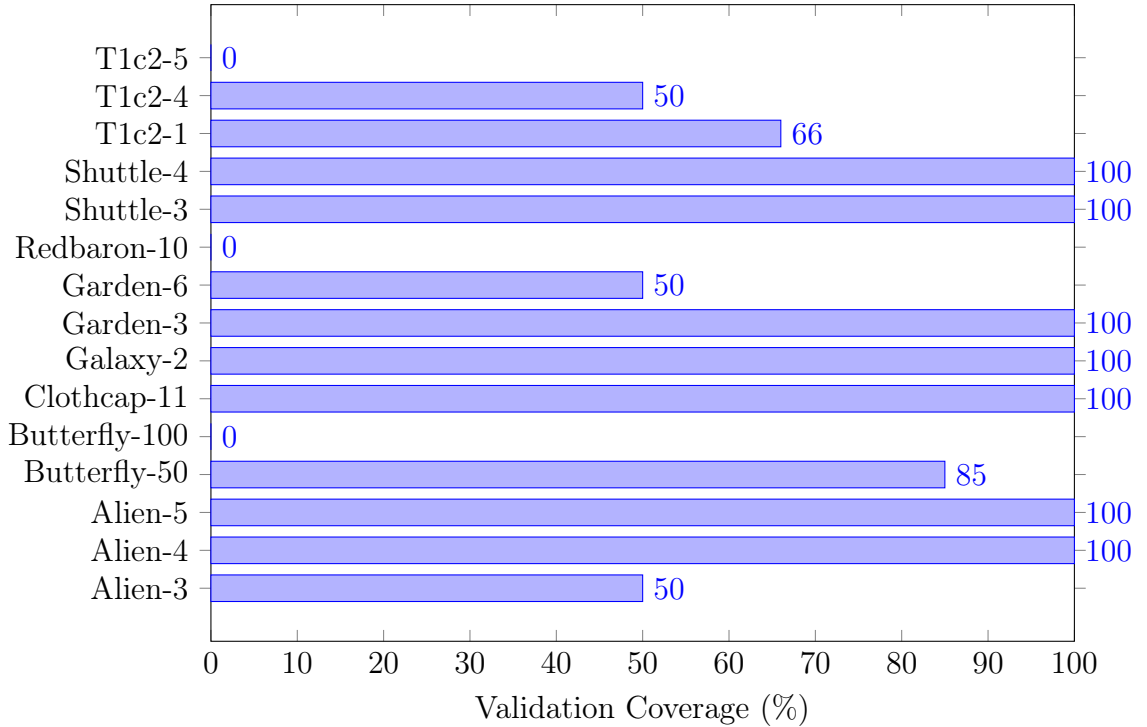


Figure 6.10: Percentage of executed validation tests for each laboratory.

The results summarized in Table 6.10 show that ROSET-Adapter successfully reconstructed the expected topology for all considered laboratories. In every scenario, the number of instantiated routers exactly matches the expected topology, demonstrating that the extraction process is robust across a heterogeneous set of network configurations. This observation is further confirmed by Figure 6.11, where the expected and instantiated topologies coincide for every evaluated laboratory.

Similarly, BGP convergence is successfully achieved in all but one scenario. The only exception is the *RedBaron* laboratory, where the adaptation process fails to generate a complete routing information base. A manual inspection of the original laboratory revealed that its router configurations follow a legacy Quagga-style syntax rather than the FRR configuration format assumed by ROSET-Adapter. As a consequence, the extraction process is unable to correctly obtain

the routing information required to generate the MRT RIB dump, preventing the emulated network from reaching the expected converged state. This limitation is therefore attributable to the current parser implementation that no longer support this style of configuration rather than to the verification framework itself.

The validation coverage shown in Figure 6.10 should also be interpreted carefully. In most cases, missing verification actions are not caused by failures of ROSE-T or ROSET-Adapter, but rather by the characteristics of the selected laboratories. Several scenarios use private or isolated provider networks that do not advertise publicly routable prefixes. Since some verification actions rely on the presence of publicly visible routing information, they cannot be meaningfully executed under these conditions.

The remaining missing validations are almost exclusively associated with the anti-spoofing verification. As discussed in the previous chapters, this action requires the framework to automatically identify one valid interface and IP address from which spoofed traffic should be generated. While the implemented heuristics are effective for the majority of scenarios, some laboratory topologies present ambiguous interface layouts that prevent the automatic selection of a suitable injection point. These cases therefore reflect the intrinsic difficulty of automatically inferring packet injection parameters from arbitrary network configurations, rather than shortcomings in the verification logic itself.

Overall, the evaluation demonstrates that the proposed pipeline is sufficiently robust to operate on laboratories that were not specifically designed for MANRS verification. The topology reconstruction process proved to be reliable across all evaluated scenarios, while the few observed limitations are primarily related to unsupported configuration formats and to assumptions required by specific verification actions. These results suggest that the approach can be successfully applied to a wide range of existing network laboratories with only limited adaptation effort.

An additional aspect evaluated during the experiments concerns the automatic

generation of compliant scenarios through ROSET-Adapter. As anticipated in the previous chapters, this functionality is intended as a best-effort remediation mechanism rather than a fully automated compliance solution. The adaptation process applies a predefined set of configuration changes, such as the insertion of routing policies and Reverse Path Forwarding (RPF) rules, with the goal of satisfying the requirements of the supported MANRS actions.

The experimental results confirm the expectations established during the design phase. In several scenarios, the automatically generated configurations successfully eliminate the detected compliance violations, allowing ROSE-T to validate the resulting network without requiring further manual intervention. In other laboratories, however, the proposed modifications are insufficient to achieve full compliance. This behaviour is expected, as routing policies are inherently dependent on the specific topology, business relationships and operational objectives of each network. Consequently, a generic transformation strategy cannot capture every possible deployment scenario without risking the introduction of incorrect routing behaviour.

Rather than replacing manual network engineering, the compliant scenario generation should therefore be regarded as a practical aid for experimentation and as a starting point for network operators. It significantly reduces the effort required to obtain a compliant configuration in many common cases while leaving the final validation and any topology-specific refinements to the operator.

To ensure reproducibility and facilitate further experimentation, all evaluated laboratories, the generated configurations, the extracted datasets and the results obtained are publicly available on ROSET-Adapter GitHub repository [Sac26c], allowing the complete evaluation presented in this chapter to be independently reproduced.

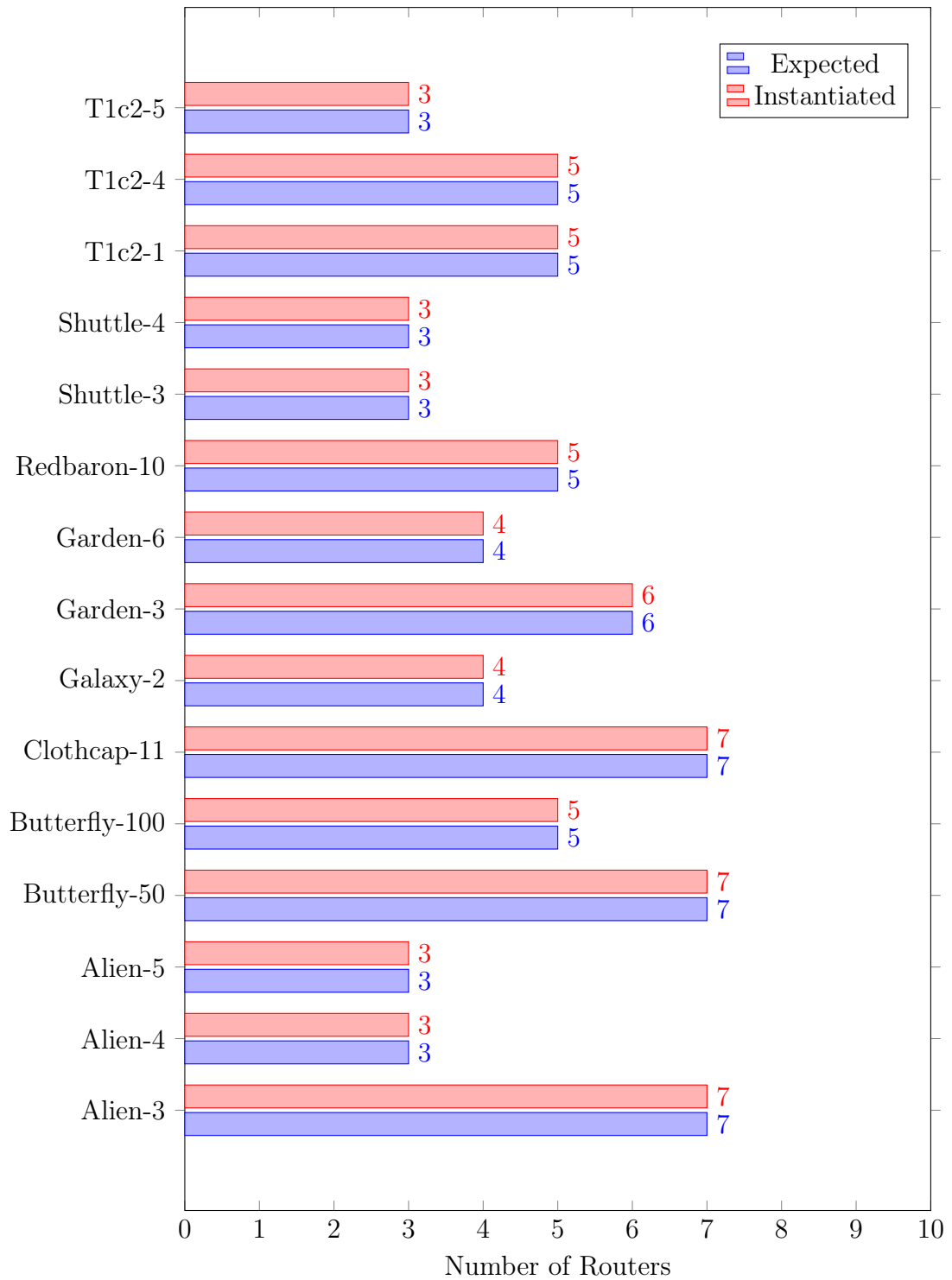


Figure 6.11: Comparison between the expected number of routers and the routers instantiated.

6.4 Summary and Discussion

Overall, the experimental evaluation demonstrates that the proposed extensions successfully achieve the objectives defined in this work. The controlled scenarios confirm that ROSE-T is able to correctly detect both compliant and non-compliant configurations according to the manually established ground truth, while the evaluation on existing Kathará laboratories shows that the proposed adaptation pipeline is sufficiently robust to operate on networks that were not originally designed for MANRS verification.

The experiments were intentionally designed to challenge the framework by selecting scenarios that expose potentially problematic situations, such as increasingly complex internal topologies, multiple customer relationships, multi-homed networks and heterogeneous router configurations. Rather than populating the evaluation with trivial test cases that would also have been successfully handled by previous versions of ROSE-T, the focus was placed on validating the newly introduced functionalities under conditions where failures or inaccuracies were considered more likely to occur. Although additional scenarios could always be considered, the selected laboratories provide a representative and sufficiently diverse set of configurations for assessing both the correctness and the robustness of the proposed approach.

A limited number of warnings were observed during the evaluation, primarily related to IPv6 support. These warnings do not affect the outcome of the verification process and are mainly due to the fact that some laboratories include partial or incomplete IPv6 configurations that are not relevant to the evaluated MANRS actions. Consequently, they should be interpreted as implementation limitations of the experimental environments rather than as shortcomings of the verification methodology.

The experiments also highlight the current limitations of the proposed framework. In particular, the automatic generation of compliant scenarios cannot guar-

antee successful remediation for every possible network configuration, as routing policies are inherently dependent on the specific operational requirements and business relationships of each Autonomous System. Similarly, some verification actions rely on assumptions that cannot always be satisfied by arbitrary laboratory topologies, such as the availability of publicly visible routing information or the automatic identification of suitable packet injection points for anti-spoofing verification.

Nevertheless, the overall results indicate that the proposed extensions substantially improve the applicability of ROSE-T. The introduction of ROSET-Adapter enables the framework to operate on existing network laboratories with minimal manual intervention, while preserving the original verification workflow. At the same time, the experimental campaign confirms that the reconstructed topologies, generated routing information and automated verification pipeline remain reliable across a broad range of realistic network scenarios, providing a solid foundation for future extensions and additional MANRS validation capabilities.

Conclusions and Future Developments

Conclusions

This thesis presented an extension of ROSE-T, an open-source framework for the automated verification of MANRS compliance through network emulation. The original framework was limited to scenarios in which a single router represented the entire candidate Autonomous System, preventing its application to realistic network deployments where multiple interconnected routers collectively implement the routing and security policies of an AS.

Four main contributions were introduced to address this limitation. The addition of FRRouting support extended the set of candidate configurations that ROSE-T can evaluate, covering one of the most widely adopted open-source routing platforms and enabling verification in fully open-source environments. The Global Information File mechanism removed the dependency on external Internet resources for controlled experimentation, allowing the creation of self-contained laboratory scenarios whose routing information is entirely under the control of the experimenter. The multi-router architecture redesign constitutes the primary contribution of this work: the topology reconstruction process was extended to instantiate and interconnect all BGP-speaking routers of the candidate AS and all verification actions were redesigned to evaluate compliance at the Autonomous

System level rather than at the individual router level. Finally, ROSET-Adapter was developed to automate the extraction of routing information from existing Kathará laboratories and the generation of all inputs required by the verification framework.

The experimental evaluation demonstrated that the extended framework correctly identifies routing policy violations and validates compliant configurations across a diverse set of laboratory scenarios. The controlled experiments confirmed the accuracy of the verification results against manually established ground truth, while the benchmark campaign showed that the topology reconstruction pipeline is sufficiently robust to operate on laboratories that were not originally designed for MANRS verification. In all evaluated scenarios, the number of reconstructed routers exactly matched the expected topology and BGP convergence was achieved in all but one case, where the failure was attributable to a legacy configuration format not supported by the current parser.

The analysis also highlighted the current limitations of the proposed approach. The Anti-Spoofing verification relies on heuristics for client placement that cannot always identify suitable attachment points in arbitrary topologies, resulting in partial validation coverage in some scenarios. Similarly, the automatic generation of compliant configurations through ROSET-Adapter cannot guarantee remediation in every possible deployment, as routing policies are inherently dependent on the operational context and business relationships of each network. These limitations reflect the fundamental challenge of automating compliance verification in a domain characterized by significant heterogeneity and operational complexity.

Overall, the proposed extensions substantially improve the applicability of ROSE-T to realistic network scenarios, providing network operators with a practical tool for evaluating the security posture of multi-router Autonomous Systems before applying configuration changes to production infrastructures.

Future Developments

Several directions for future work emerge from the limitations and observations discussed throughout this thesis.

RPKI support. The current implementation of the Route Leak verification focuses exclusively on valley-free routing policy enforcement and does not evaluate prefix filtering based on Resource Public Key Infrastructure (RPKI) Route Origin Authorizations (ROAs). Integrating RPKI validation into the verification pipeline would extend the coverage of the Filtering action to include origin authentication, providing a more complete assessment of routing security.

Extended MANRS+ coverage. The MANRS initiative has continued to evolve since the original design of ROSE-T, introducing additional requirements and extending its scope beyond the four original actions. Future versions of the framework could incorporate verification of additional MANRS requirements, including support for the validation criteria defined in the MANRS+ program aimed at more advanced routing security practices. In the official website [MAN25] can be found more information about MANRS+.

Improved client placement heuristics. The Anti-Spoofing verification currently relies on conservative heuristics that skip the validation when a suitable client attachment point cannot be identified automatically. More sophisticated placement strategies, potentially leveraging a deeper analysis of the advertised prefixes and the internal routing topology, could increase the validation coverage without compromising the reliability of the results.

Loopback-based iBGP support. The current topology reconstruction process does not support loopback-based iBGP peering, which is one of the most common configurations in operational networks. Extending the framework to correctly model multi-hop iBGP sessions through loopback addresses would significantly improve the fidelity of the reconstructed topologies and reduce the reliance on the hub-based approximation currently adopted.

Scalability analysis. The experimental evaluation showed that execution time grows with the number of candidate routers and provider relationships, primarily due to the Anti-Spoofing verification. A systematic scalability analysis, combined with optimizations such as parallel execution of independent validation tests, could make the framework suitable for larger operational deployments.

Validation on operational networks. Since the original version of ROSE-T has already been evaluated on real Autonomous System configurations, a natural continuation of this work would be to repeat the same validation campaign using the extended framework. This would make it possible to assess the impact of the proposed architectural changes in real operational environments while confirming that the additional functionality preserves the effectiveness of the original approach. Such an evaluation would require the collaboration of network operators willing to provide the configurations of all border routers belonging to the candidate AS, as the multi-router verification process depends on a complete view of the routing infrastructure.

Bibliography

- [BB05] Robert Beverly and Steven Bauer. The spoofer project: inferring the extent of source address filtering on the internet. pages 8–8, 07 2005.
- [BGO⁺16] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *Communications of the ACM*, 59(5):50–57, 2016.
- [BILDB18] Gaetano Bonofiglio, Veronica Iovinella, Gabriele Lospoto, and Giuseppe Di Battista. Kathará: A container-based framework for implementing network function virtualization and software defined networks. In *NOMS 2018 - 2018 IEEE/IFIP Network Operations and Management Symposium*, pages 1–9, 2018.
- [Bio07] Philippe Biondi. Scapy. In *Proceedings of the CanSecWest Conference*, Vancouver, Canada, 2007.
- [BS04] Fred Baker and Pekka Savola. Ingress Filtering for Multihomed Networks. RFC 3704, March 2004.
- [FFP⁺15] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd D. Millstein. A general approach to network configuration analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation, NSDI*

- 15, Oakland, CA, USA, May 4-6, 2015, pages 469–483. USENIX Association, 2015.
- [Fra26] Kathará Framework. Kathará labs. <https://github.com/KatharaFramework/Kathara-Labs>, 2026. GitHub repository, accessed June 2026.
- [L⁺16] Kristian Larrson et al. vrnetlab: Run virtual routers with docker. <https://github.com/vrnetlab/vrnetlab>, 2016. Accessed: 2026-06-21.
- [MAN] <https://manrs.org/about/>. MANRS official website.
- [MAN25] MANRS+ Working Group. MANRS+ Concept. <https://manrs.org/about/manrs-working-group/manrs-concept/>, 2025. Accessed: 2026-06-26.
- [Mer14] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. In *Proceedings of the Linux Symposium*, volume 2, pages 239–246, 2014.
- [Mey03] David M Meyer. The oregon route-views project. *NSF Award Number 0323769. Directorate for Computer and Information Science and Engineering*, 3(323769):23769, 2003.
- [MKR⁺96] Robert Moskowitz, Daniel Karrenberg, Yakov Rekhter, Eliot Lear, and Geert Jan de Groot. Address Allocation for Private Internets. RFC 1918, February 1996.
- [MSPa] Tommaso Caiazzi Mariano Scazzariello and Antonio Prado. <https://blog.apnic.net/2024/05/20/verify-manrs-compliance-automatically-with-rose-t/>. Roset presentation.

- [MSPb] Tommaso Caiazzì, Mariano Scazzariello, and Antonio Prado. <https://github.com/KatharaFramework/roset>. Roset presentation.
- [MSW⁺13] Prodosh Mohapatra, John Scudder, David Ward, Randy Bush, and Rob Austein. BGP Prefix Origin Validation. RFC 6811, January 2013.
- [SAC20] Mariano Scazzariello, Lorenzo Ariemma, and Tommaso Caiazzì. Kathará: A lightweight network emulation system. In *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, pages 1–2, 2020.
- [Sac26a] Lorenzo Sacco. Rose-t: Extended multi-router version. <https://github.com/lorenzoscacco02/roset>, 2026. GitHub repository, accessed June 2026.
- [Sac26b] Lorenzo Sacco. Roset-adapter. <https://github.com/lorenzoscacco02/ROSET-Adapter>, 2026. GitHub repository, accessed June 2026.
- [Sac26c] Lorenzo Sacco. Roset tests. <https://github.com/lorenzoscacco02/ROSET-Adapter/tree/main/Examples>, 2026. GitHub repository, accessed June 2026.
- [SADBP21] Mariano Scazzariello, Lorenzo Ariemma, Giuseppe Di Battista, and Maurizio Patrignani. Megalos: A scalable architecture for the virtualization of large network scenarios. *Future Internet*, 13(9), 2021.
- [SF00] Daniel Senie and Paul Ferguson. Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing. RFC 2827, May 2000.
- [SKG⁺18] Pavlos Sermpezis, Vasileios Kotronis, Petros Gigis, Xenofontas Dimitropoulos, Danilo Cicalese, Alistair King, and Alberto Dainotti.

- Artemis: Neutralizing bgp hijacking within a minute. *IEEE/ACM transactions on networking*, 26(6):2471–2486, 2018.
- [SMH20] Kotikalapudi Sriram, Doug Montgomery, and Jeffrey Haas. Enhanced Feasible-Path Unicast Reverse Path Forwarding. RFC 8704, February 2020.
- [SMM⁺16] Kotikalapudi Sriram, Doug Montgomery, Danny R. McPherson, Eric Osterweil, and Brian Dickson. Problem Definition and Classification of BGP Route Leaks. RFC 7908, June 2016.

Ringraziamenti

Sarò breve, ma ci tengo a dedicare qualche parola a chi mi ha accompagnato durante questo bellissimo percorso.

Per prima cosa, ringrazio il mio relatore Tommaso, che è sempre stato molto disponibile e comprensivo. Sono veramente soddisfatto del lavoro che abbiamo svolto assieme e del rapporto che si è creato innanzitutto al livello umano.

Ringrazio anche Mariano e Antonio per avermi seguito durante il percorso di tesi, apprezzo molto la leggerezza con cui si sono svolti tutti i nostri incontri.

Grazie a tutti voi, lavorare a questo progetto è stato un piacere e non un impegno da portare a termine il più velocemente possibile.

Grazie a Lorenzo per aver ostacolato (senza successo) la mia laurea con Cyber-Challenge. Scherzi a parte, grazie per avermi insegnato tanto e per credere in me.

Un grazie va a tutta la famiglia, non penso che nessuno di voi abbia effettivamente ancora capito cosa studio ma, a modo vostro, avete sempre provato ad aiutarmi senza farmi mancare nulla. A volte posso sembrare indifferente a tutto questo ma senza di voi ora non sarei qui.

Grazie agli amici del liceo, ancora qui con me dopo tutti questi anni. Se volete i ringraziamenti fatti bene, leggete quelli della triennale, ora non sono in vena di smancerie. Sappiate però che sono sinceramente contento di avervi ancora nella

mia vita.

Grazie ovviamente ai compagni di Guacamole, non potevo chiedere persone migliori con cui condividere questo percorso. So che, prima o poi, scapperete tutti in giro per il mondo, ma allo stesso modo sono convinto che rimarremo legati anche da lontano, tanto voi Discord lo sapete usare.

Grazie anche a tutte le persone conosciute in università in questi ultimi due anni, fra esami e caffè abbiamo studiato (poco) e ci siamo divertiti (tanto).

Grazie anche a tutti gli amici di CyberChallenge, insieme abbiamo faticato ma abbiamo anche imparato molto, è stata una bellissima avventura. Certo... sotto al NOP non ce lo meritavamo.

Ringrazio anche i miei compagni di calcio e affini: è merito vostro se sono ingegnere soltanto a metà, nonostante le mie 10 lauree.

Ringraziamento speciale va a Claro e Joao, tornati dalla Calabria con furore. vi voglio bene e grazie per esserci sempre.

Non posso concludere senza ringraziare la mia Stellina. So che odi queste esternazioni pubbliche e, per questa volta, lo risperrerò, quindi grazie per esserci e per supportarmi sempre, Ti amo.

Per concludere posso, senza dubbio, affermare... che non sono stato così breve come avevo promesso.

